

★ WEEK 9 · PART 1 of 3

The Kalman Filter: Predict, Then Update

The Recursion That Flew to the Moon

The predict step · the update step · the innovation · one cycle per observation

Week 8 gave the state space model and the intuition of a predict-update loop. Now we make it precise: the two-step recursion that estimates the hidden state optimally, in real time.

1

PART 1 · PREDICT & UPDATE

Two Steps, Repeated Forever

The predict step that moves the state forward, the update step that corrects it with new data, and the innovation — the surprise — that drives every correction.

The Filter That Flew to the Moon — Part 1

From a doubted paper to Apollo navigation

In 1960, Rudolf Kalman published a recursive solution to estimating a hidden state from noisy measurements. Parts of the field were skeptical — until NASA saw what it could do.

The idea

- A recursive filter: update an estimate as each measurement arrives.
- No need to re-solve over the whole history each time.
- Elegant, fast, and exactly suited to real-time systems.

The turning point

- Stanley Schmidt at NASA Ames grasped its importance.
- He adapted it for the Apollo guidance computer.
- It estimated spacecraft position and velocity from noisy sensors.

The Filter That Flew to the Moon — Part 2

Why it worked, and the lesson

The filter fused an imperfect motion model with noisy measurements to produce the best possible estimate — with limited 1960s computing. The same recursion now runs in finance.

Why it worked

- It weighted the model and the data by their uncertainties.
- It carried a running estimate of how sure it was.
- It was recursive — cheap enough to run onboard, live.

The lesson for us

- The same problem is everywhere: a hidden state, noisy data.
- Trend, volatility, and hedge ratios are hidden states.
- One recursion estimates them all, optimally and in real time.

We Have the Model — Now We Need the State

Where Week 8 left off

1

Week 8 gave the framework

State space: a hidden state evolving over time, seen only through noisy observations. We even sketched a predict-update loop.

2

But we never wrote the optimal recipe

How exactly do you move the state forward, and how much do you correct it when data arrives? Week 8 gave intuition; Week 9 gives the equations.

3

The Kalman filter is that recipe

It is the optimal recursive estimator for a linear-Gaussian state space model — the precise engine behind everything in Part II.

Part 1 in one line: two steps — predict then update — repeated once per observation.

The Big Idea: Predict, Then Update

Two steps, repeated for every observation

The filter alternates two moves. PREDICT: use the model to guess the next state, becoming less certain. UPDATE: use the new observation to correct that guess, becoming more certain.

Predict (time update)

- Push the state forward by its own dynamics.
- Uncertainty grows — you know a little less.
- This gives a prediction of the next observation.

Update (measurement update)

- Compare the prediction to the actual data.
- Correct the state toward the data.
- Uncertainty shrinks — the observation informed you.

What the Filter Carries Forward

Two things, and only two

carry forward: $(\hat{x}_{t-1|t-1}, P_{t-1|t-1}) =$ state estimate and its covariance

The estimate

- The current best guess of the hidden state.
- It summarizes everything seen so far.
- It is all you need to move to the next step.

The covariance

- How uncertain that estimate is.
- It grows in predict, shrinks in update.
- It is what lets the filter weigh model against data.

The Predict Step (Time Update)

Move the state and its uncertainty forward

$$\text{Predict: } \hat{x}_{t|t-1} = T \hat{x}_{t-1|t-1}, \quad P_{t|t-1} = T P_{t-1|t-1} T' + Q$$

The state moves

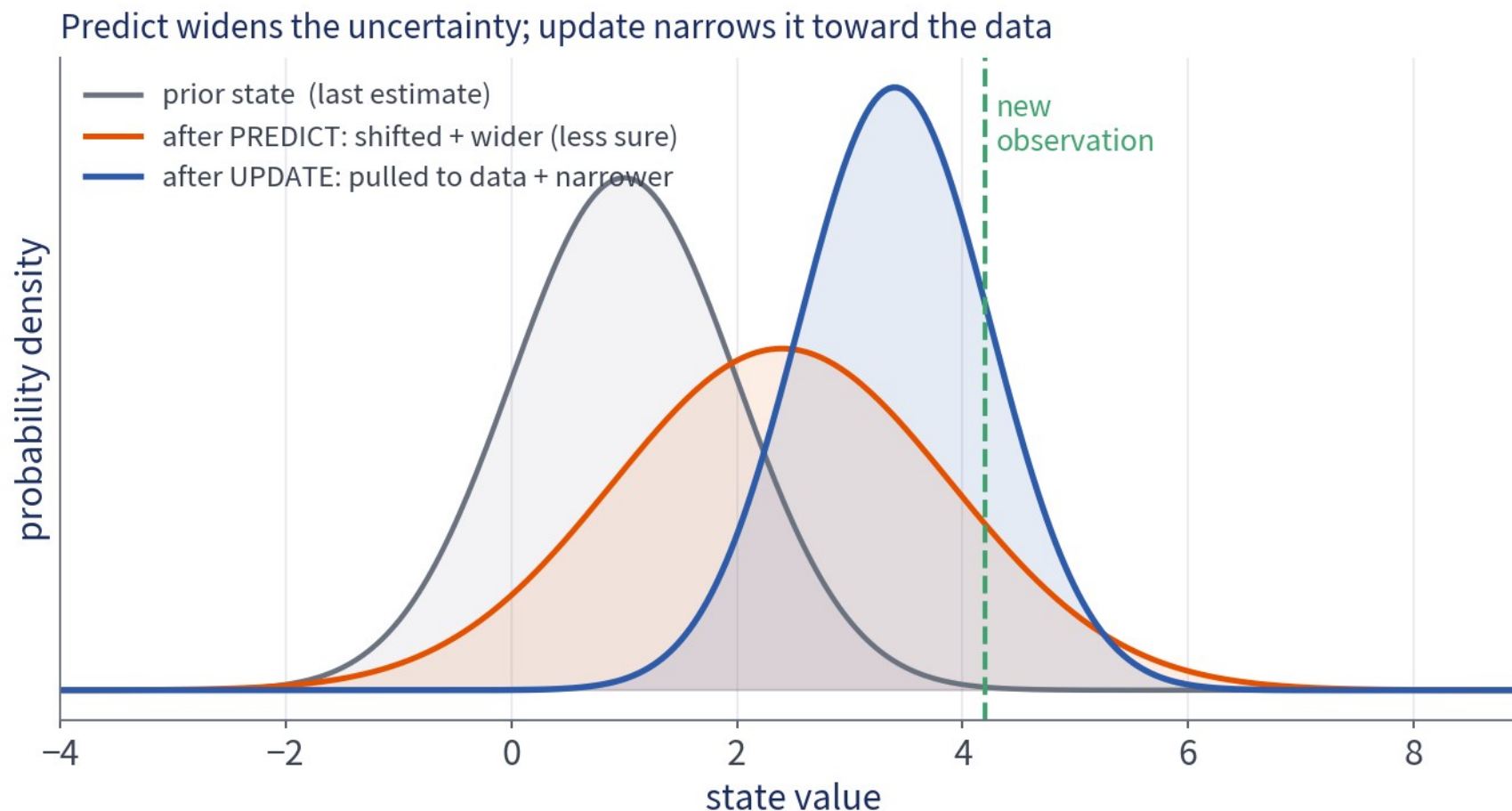
- Apply the transition T to the last estimate.
- The state coasts forward by its own dynamics.
- No data is used yet — this is a pure forecast.

The uncertainty grows

- The covariance is pushed through T and inflated by Q .
- Q is the process noise — the model isn't perfect.
- So you become LESS certain until data corrects you.

Predict Widens, Update Narrows

As a distribution over the state: predicting shifts it forward and widens it (less sure); then the new observation pulls it over and sharpens it (more sure). One full cycle.



Predicting the Next Observation

What we expect to see, and how unsure we are

$$\text{predicted obs } \hat{y}_t = Z \hat{x}_{t|t-1}; \quad \text{innovation variance } F_t = Z P_{t|t-1} Z' + H$$

The predicted observation

- Map the predicted state through Z to expect a data value.
- This is what the model says should happen next.
- Comparing it to reality is the whole point.

Its uncertainty F

- F combines state uncertainty and measurement noise H .
- It says how surprised we are ALLOWED to be.
- It scales the correction in the update step.

The Innovation: The Surprise

The signal that drives every correction

$$\text{Innovation (surprise): } v_t = y_t - \hat{y}_t = y_t - Z \hat{x}_{t|t-1}$$

What it is

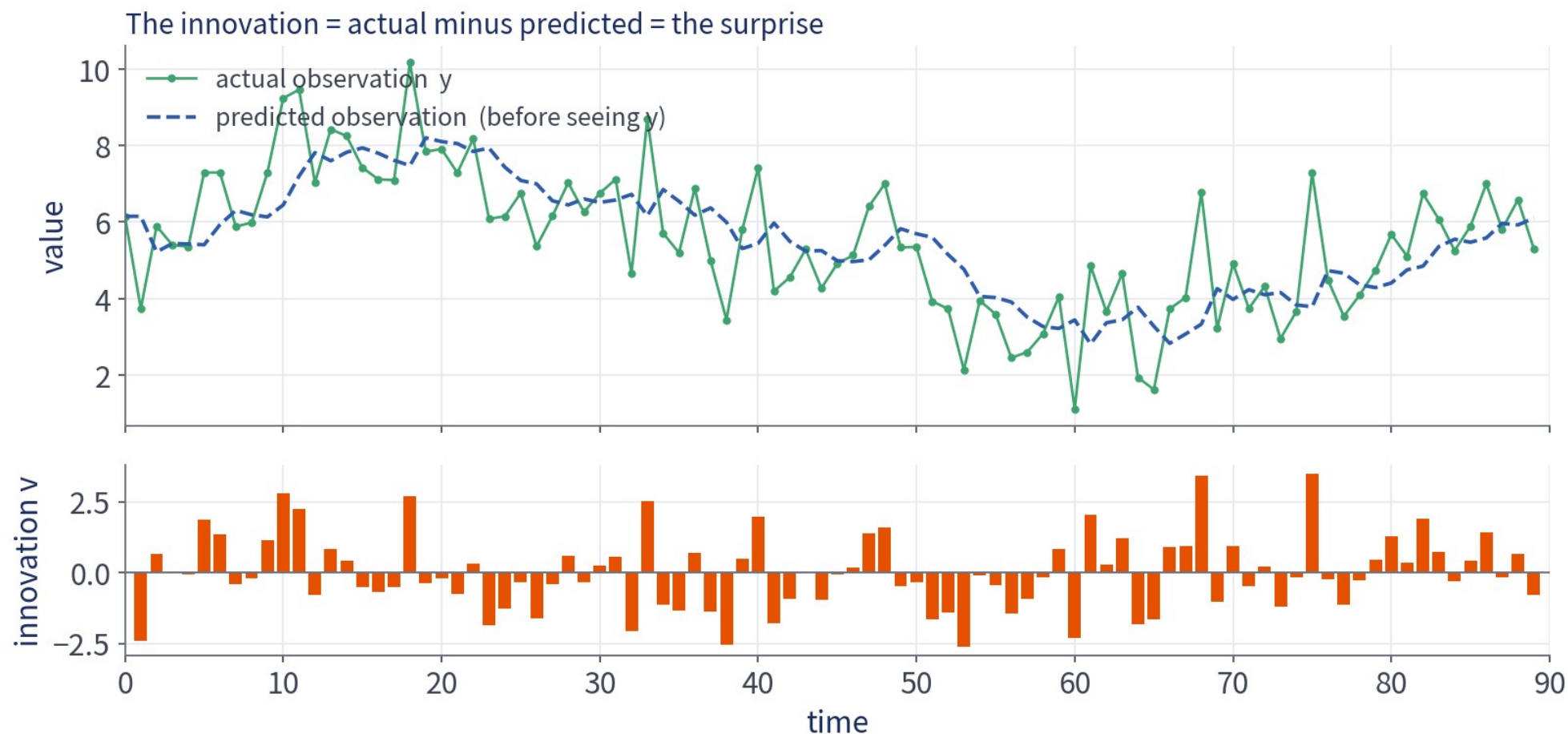
- The gap between the actual and predicted observation.
- How wrong the model's forecast turned out to be.
- The only NEW information the observation carries.

Why it is central

- The correction is built entirely from the innovation.
- No surprise $\rightarrow$ no correction needed.
- In a good model, the innovations are just white noise.

The Surprise, Each Step

Before seeing each point, the filter predicts it (dashed). The gap to the actual value (top) is the innovation (bottom) — the surprise that the update step will act on.



The Update Step (Measurement Update)

Correct the prediction with the new data

Update: $\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t$ (correction = gain $\times$ surprise)

The correction

- Add the gain times the innovation to the predicted state.
- The correction points toward what the data implies.
- The gain K sets how big that step is (Part 2).

The result

- A new, better estimate that has absorbed the observation.
- The covariance shrinks — you are more certain now.
- This estimate becomes the input to the next predict.

Correction Proportional to Surprise

Bigger surprise, bigger correction

The rule

- The correction is the gain multiplied by the innovation.
- A large surprise moves the estimate a lot.
- A tiny surprise barely moves it at all.

Why this is sensible

- If the data matches the model, leave the estimate alone.
- If the data is far off, pay attention and adjust.
- The gain tunes exactly how much attention to pay.

The Predict-Update Cycle

One pass per observation, forever

Predict, then update, then advance one step and repeat. Each observation triggers exactly one cycle. The estimate and its uncertainty carry from one cycle to the next — no refitting.

The loop

- Predict the state and grow its uncertainty.
- Observe, compute the innovation, and update.
- Advance to the next time step and repeat.

Why it is beautiful

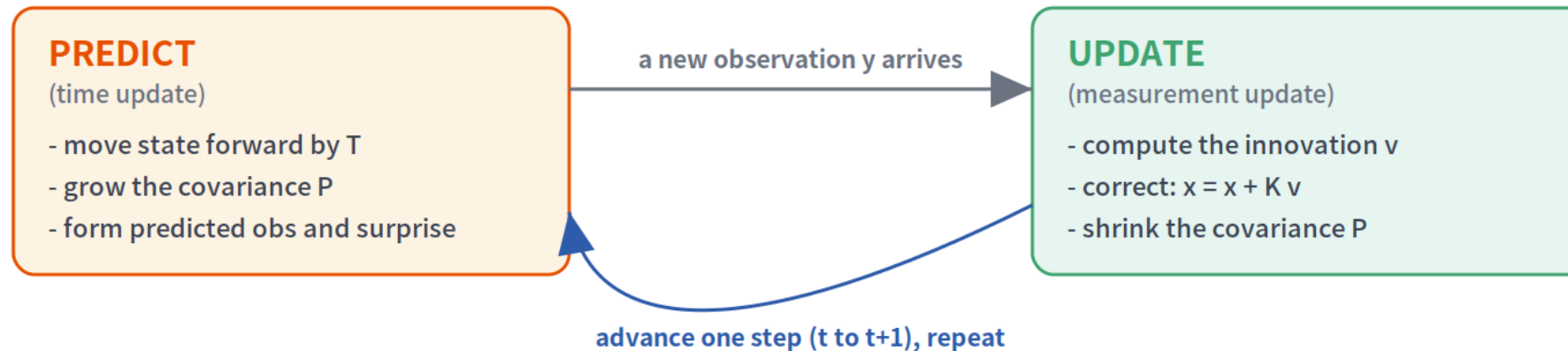
- Constant work per step — it never slows down.
- It handles streaming data naturally.
- The whole past is summarized in two carried objects.

The Predict-Update Cycle

Guess ahead (predict), let a new observation reveal the surprise, correct by the gain times the innovation (update), then advance and repeat. Part 2 makes the correction optimal.

The Predict-Update Cycle

One pass per observation, forever: guess ahead, measure the surprise, correct.



The surprise drives everything

The correction is the Kalman gain K times the innovation v (how wrong the prediction was).

Big surprise or high state uncertainty $\rightarrow$ large correction. Small surprise or precise state $\rightarrow$ barely move.

Part 2 derives the gain K that makes this correction optimal.

The Full Recursion at a Glance

Five equations, two steps - the whole filter on one slide

PREDICT	$\hat{x}_{t t-1} = T \hat{x}_{t-1 t-1}$	$P_{t t-1} = T P_{t-1 t-1} T' + Q$
innovation	$v_t = y_t - Z \hat{x}_{t t-1}$	$F_t = Z P_{t t-1} Z' + H$
gain		
UPDATE	$\hat{x}_{t t} = \hat{x}_{t t-1} + K_t v_t$	$P_{t t} = (I - K_t Z) P_{t t-1}$

Predict (time update)

- Move the state forward by T; inflate its covariance by Q.
- No data is used yet - a pure model forecast.
- You become LESS certain.

Update (measurement update)

- Form the innovation v and its variance F.
- The gain K weighs model vs data; correct by K times v.
- Shrink the covariance - you become MORE certain.

A Single Step, Walked Through

Following one observation through the filter

Predict

- Start from last estimate 10.0 with some uncertainty.
- Model says the level is roughly unchanged: predict 10.0.
- Uncertainty grows a little (process noise added).

Update

- The new observation comes in at 12.0 → surprise = +2.0.
- With a gain of about 0.4, correct by $0.4 \times 2.0 = 0.8$.
- New estimate 10.8, with smaller uncertainty. Repeat.

Filtering in Real Time

Built for streaming data

The recursive advantage

- Each new point costs the same, small amount of work.
- You never re-process the whole history.
- The estimate is always current, the instant data lands.

Where this matters

- Live trading signals and risk estimates.
- High-frequency and streaming data.
- Anywhere decisions can't wait for a batch re-fit.

Initializing the Recursion

Every recursion needs a first guess

The starting point

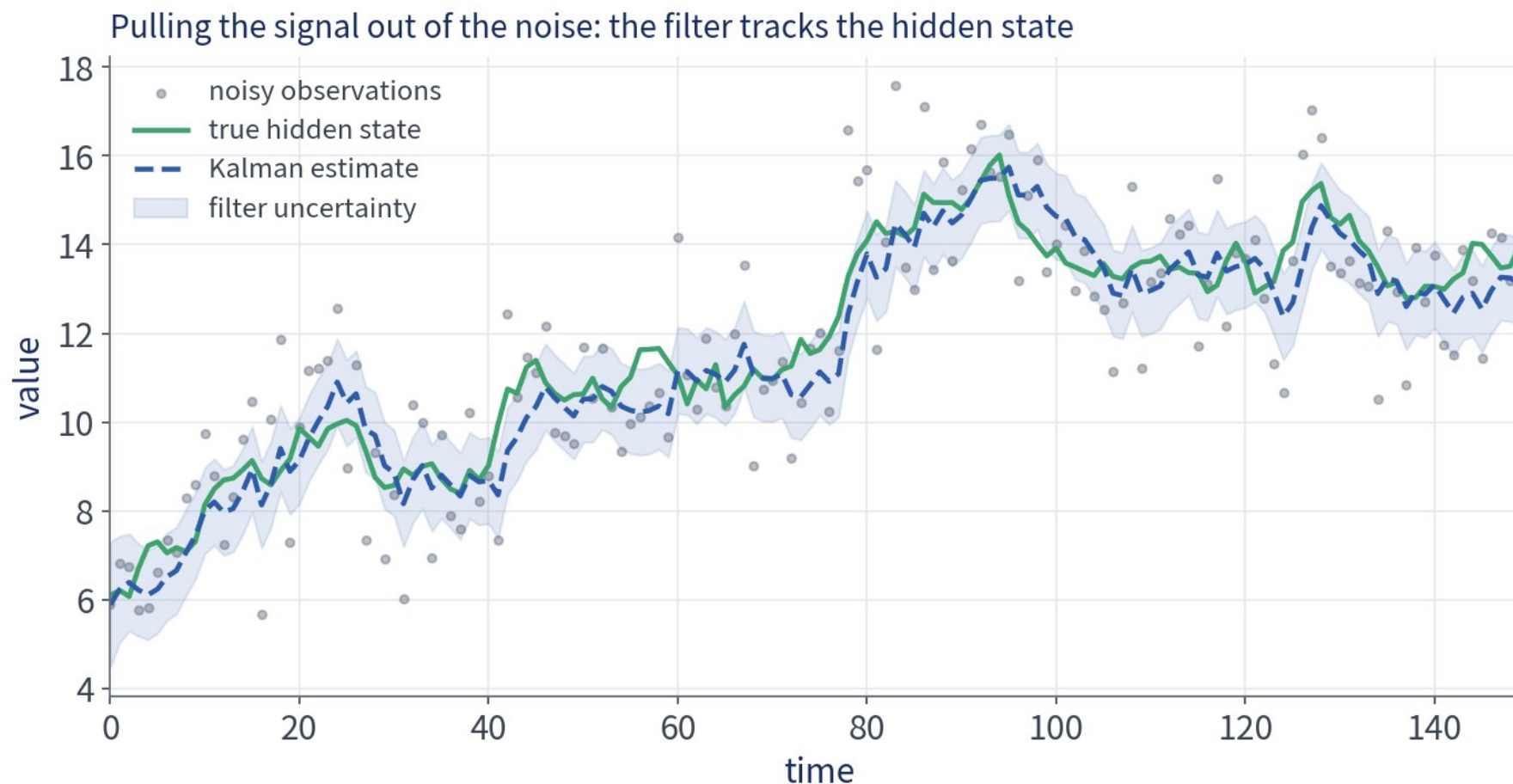
- Begin with a prior estimate and a prior covariance.
- If you know little, start with a large covariance.
- That tells the filter to trust early data heavily.

What happens next

- A vague prior is quickly overwritten by the data.
- The estimate converges as observations accumulate.
- A poor start washes out — the filter self-corrects.

Pulling the Signal Out of the Noise

Noisy observations go in; a clean estimate of the hidden state comes out, with an uncertainty band. This denoising is the filter's most visible payoff.



Recursive vs Batch Estimation

Update, don't re-solve

The batch way

- Re-estimate over the entire dataset each time.
- Cost grows with the history — soon infeasible live.
- Wasteful: most of the answer hasn't changed.

The recursive way

- Carry a compact summary and update it.
- Constant cost per step, forever.
- Mathematically equivalent, but vastly cheaper.

What Parts 2 and 3 Add

From mechanism to mastery

Part 2 — the gain

- The exact formula for the Kalman gain K .
- How it balances trust in the model vs the data.
- The full five-equation recursion, and when it is optimal.

Part 3 — applications

- Denoising and a dynamic, drifting regression beta.
- Pairs trading Stage III: a self-adjusting hedge ratio.
- Practical cautions and the bridge to hidden states (Week 10).

Implement Predict-Update by Hand

Direct Claude Code to build the cycle from scratch

GOAL Code a scalar Kalman filter yourself to feel the two steps — then check it against a library.

[INPUT] Prompt · Claude Code

Write and run `kalman_by_hand.py` that:

1. simulates a local level: a hidden random-walk state plus measurement noise;
 2. implements the filter as an explicit loop with a PREDICT step and an UPDATE step (no library);
 3. at each step prints the innovation and the correction (gain x innovation);
 4. plots the observations, the true state, and the filtered estimate with its uncertainty band;
 5. cross-checks against `statsmodels`' filter.
- Show that predict grows P and update shrinks it.

How to work with it

- Watch P rise in predict, fall in update.
- The innovation should look like white noise.
- Your loop should match the library closely.
- Keep it; Part 2 adds the optimal gain.

Deliverable: `kalman_by_hand.py` + a filtered-state plot and a library cross-check.

Three Questions Before You Move On

If you can answer these, Part 1 has landed

Q1	What happens to the covariance P in the predict step vs the update step?	Grows, then shrinks
Q2	What exactly is the innovation?	Actual minus predicted obs
Q3	In the update, the correction equals what times what?	Gain times innovation

All three come from the two steps, the surprise, and the correction rule.

Three Misconceptions to Avoid

Where Kalman intuition slips

Common myths

- "The filter just smooths the data."
- "It re-processes all past data each step."
- "Predict makes you more certain about the state."

The reality

- It fuses a model with data, weighted by uncertainty.
- It is recursive — only a compact summary is carried.
- Predict makes you LESS certain; only data (update) sharpens it.

Three Things to Carry Forward

Part 1 in one screen

1

Two steps, repeated forever

Predict moves the state forward and grows its uncertainty; update corrects it with new data and shrinks the uncertainty. One cycle per observation, recursively.

2

The innovation drives every correction

The surprise — actual minus predicted observation — is the only new information a data point brings. The update is a correction built entirely from it.

3

The correction is gain times surprise

How much you move depends on the Kalman gain, which weighs the model against the data. Deriving that optimal gain is the subject of Part 2.

Next — Part 2: the Kalman gain, the covariance, the full recursion, and when it is optimal.

★ WEEK 9 · PART 1 · PRACTICE

Predict and Update, Worked by Hand

Follow One Surprise Through the Filter; Read the Innovation

The predict-update cycle, made concrete on a tracked price

Part 1's mechanics, with the arithmetic shown. We walk one full predict-update iteration by hand - predicted observation, innovation, gain, correction - then use the standardized innovation to flag news and broken models.

1P

PART 1 · APPLICATIONS

The Surprise That Runs the Filter

Case A: one full predict-update iteration on a tracked fair value, with the innovation and its variance made explicit.
Case B: the standardized innovation as a signal - flagging a four-sigma surprise. Then a Claude Code lab.

What Part 1 Told Us

Now we run one step - with numbers

1

Predict, then update

Move the state and its uncertainty forward, then correct them when the new observation arrives. Two steps, repeated forever.

2

The innovation is the surprise

The gap between what you observed and what you predicted. It drives every correction - no surprise, no update.

3

The gain weighs the surprise

How much to trust the surprise depends on the prediction and measurement uncertainty. We compute one full step by hand.

Everything here is a calculator exercise: one prior, one print, one surprise - then four of them.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from a prior state and its variance.
- Compute predict, innovation, and update by hand.
- Then read what the surprise is telling you.

What you should watch

- How the gain splits the difference to the print.
- How the innovation variance sets the gain.
- How a big standardized innovation raises a flag.

One Predict-Update, Start to Finish

Follow a single surprise through the filter

A desk tracks a fair value that drifts. A new print arrives. We walk the full cycle by hand: predict the state and its variance forward, predict the observation, measure the innovation and its variance, form the gain, and correct the estimate. Every piece of the Kalman filter in one step

The ingredients

- Prior state 100.0, variance 0.20.
- Process (drift) variance $Q = 0.05$.
- Measurement noise $R = 0.30$.

What we'll do by hand

- Predict the state and grow the variance.
- Compute the innovation and its variance.
- Form the gain and update the state.

The Predict Step

Move the state and uncertainty forward

predict state: $x^- = x = 100.0$

predict variance: $P^- = P + Q = 0.20 + 0.05 = 0.25$

With no new data yet, the state stays at 100.0 but its variance grows by the drift Q - the filter gets less certain as time passes.

The Innovation and Its Variance

The surprise, and how surprising it is

$$\begin{aligned} \text{predicted obs: } \hat{y} = x^- &= 100.0, & \text{actual } y &= 100.8 \\ \text{innovation: } \nu = y - \hat{y} &= +0.8 & & \text{(the surprise)} \\ \text{innovation var: } S = P^- + R &= 0.25 + 0.30 = 0.55 \end{aligned}$$

The print beat the prediction by 0.8 - that's the innovation. Its variance $S = 0.55$ combines how unsure we were with how noisy the print is.

The Update Step

Correct the state by the gain

$$\text{gain: } K = \frac{P^-}{S} = \frac{0.25}{0.55} = 0.455$$

$$\text{update state: } x = x^- + K \nu = 100.0 + 0.455(0.8) = \mathbf{100.36}$$

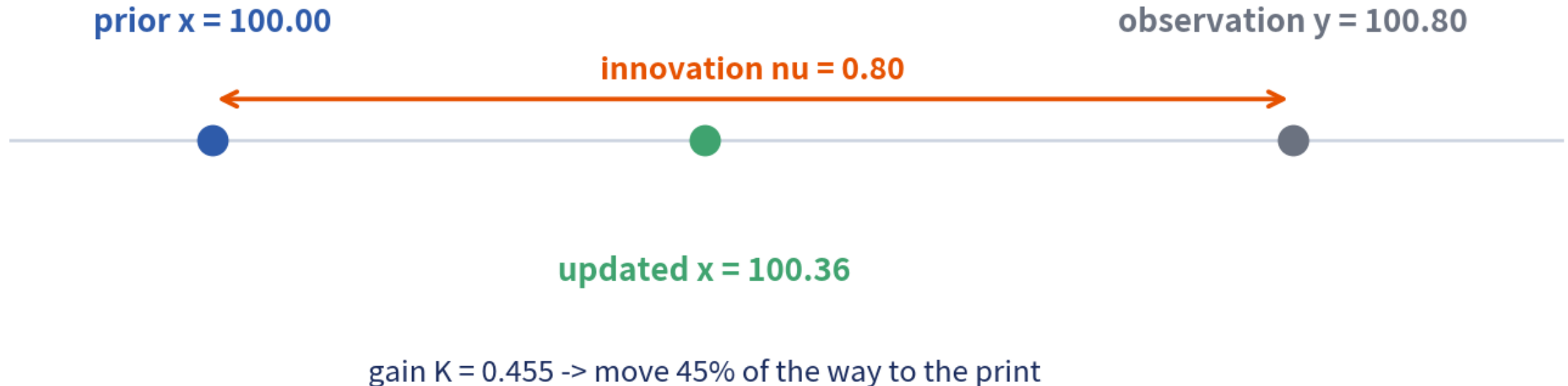
$$\text{update variance: } P = (1 - K) P^- = 0.14$$

The gain 0.455 moves the estimate 45% of the way to the print - to 100.36 - and the variance shrinks to 0.14. The surprise has been absorbed.

One Iteration, on a Line

The prior sits at 100.00 and the print at 100.80 - an innovation of 0.80. The gain of 0.455 lands the updated estimate at 100.36, about 45% of the way to the print. A bigger gain would trust the print more; a smaller one, less.

Case A: one predict-update - the innovation pulls the estimate part-way to the print



The Innovation Is the News

Every update is driven by surprise

No surprise, no update

- If the print equals the prediction, the innovation is zero.
- The gain has nothing to act on.
- The estimate doesn't move.

The surprise carries the information

- Only the unexpected part of a print is new.
- The gain decides how much of it to believe.
- That is the whole logic of the filter.

The Innovation as a Signal

When a surprise is too big to ignore

The innovation isn't just fuel for the update - it's information in its own right. Standardize it by its own standard deviation, and you get a z-score for surprise. A normal-sized one is business as usual; a four-sigma one means real news, or a model that has broken. We track four steps by hand

The idea

- Standardize the innovation: $z = nu / \sqrt{S}$.
- It measures surprise in standard deviations.
- Most steps sit comfortably within ± 2 .

What we'll do by hand

- Run four predict-update steps.
- Compute the standardized innovation each time.
- Flag the one that breaks the ± 2 band.

The Standardized Innovation

Surprise, measured in sigmas

$$z = \frac{\nu}{\sqrt{S}} = \frac{0.8}{\sqrt{0.55}} = 1.08 \Rightarrow \text{a normal-sized surprise } (|z| < 2)$$

How to read it

- z near zero: the print matched the model.
- $|z|$ up to 2: normal, expected noise.
- $|z|$ above 2: a genuinely large surprise.

Why standardize

- Raw innovations aren't comparable over time.
- S changes as the filter's certainty changes.
- Dividing by $\text{sqrt}(S)$ makes them a clean z -score.

Four Steps of Surprise

Standardized innovation each step

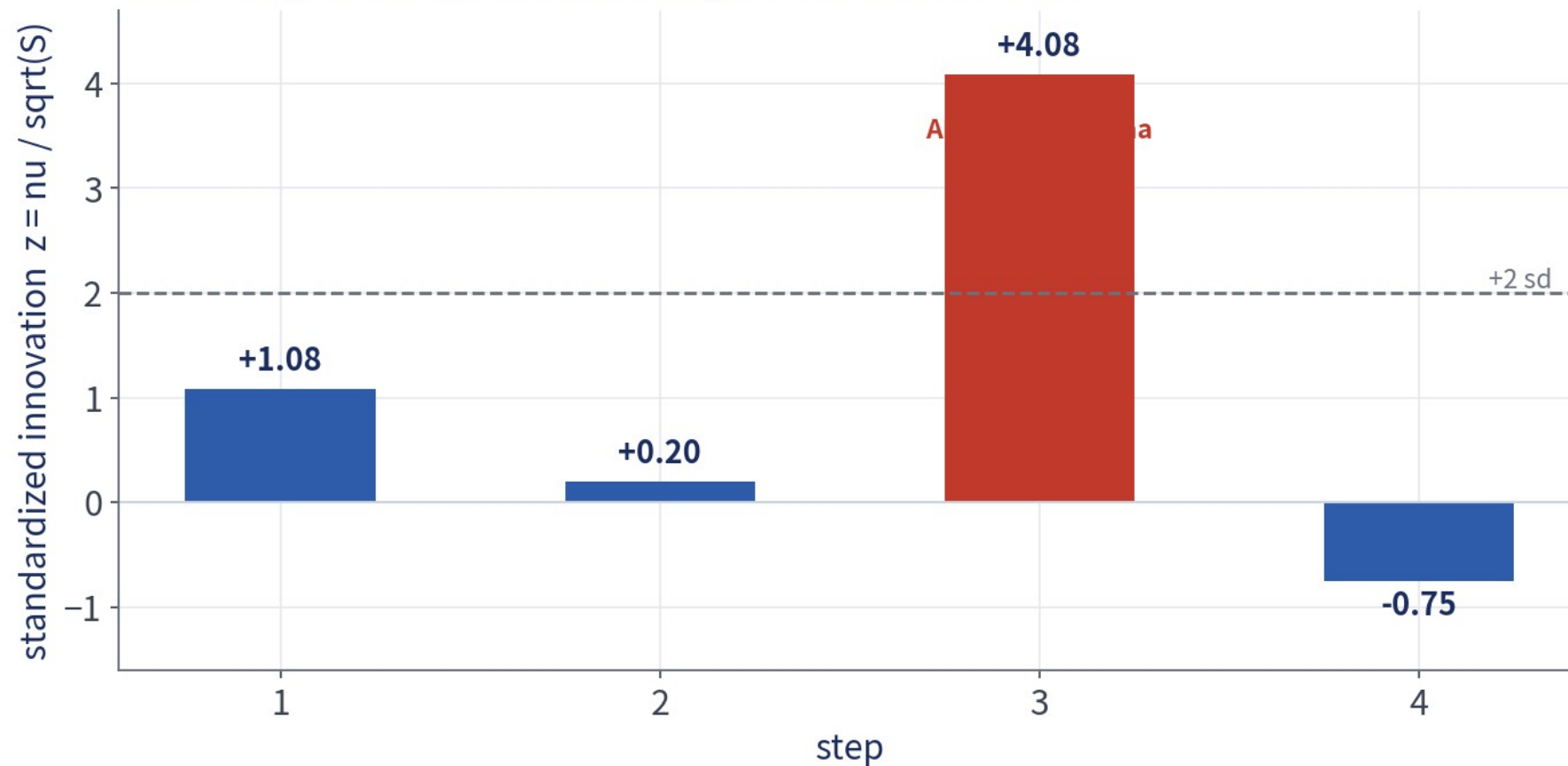
step	print y	innovation nu	S	$z = \text{nu} / \text{sqrt}(S)$
1	100.8	+0.80	0.55	+1.08
2	100.5	+0.14	0.49	+0.20
3	103.2	+2.78	0.47	+4.08 ALARM
4	100.9	-0.50	0.46	-0.75

Steps 1, 2, and 4 are ordinary. Step 3's print of 103.2 is a four-sigma surprise - far beyond what the filter expected.

A Four-Sigma Surprise

Three of the four standardized innovations sit inside the ± 2 band - normal noise. Step 3 spikes to +4.08, far outside it. That is the filter telling you something real happened: a news event, a data error, or a state that has genuinely jumped

Case B: a large standardized innovation flags news or a broken model



What a Big Surprise Means

News, error, or a broken model

As an alarm

- A large $|z|$ flags a structural break or news.
- Or a bad data point that needs checking.
- The filter's own math raised the flag.

As a signal

- On a mean-reverting spread, a big z can be noise.
- If so, fade it - trade against the surprise.
- Either way, the innovation is telling you to act.

The Common Pattern

The surprise runs everything

Both cases turn on the innovation. In the update, it's the fuel: the estimate moves only because the print surprised us, scaled by the gain. As a signal, its standardized size tells us whether the surprise is ordinary noise or something that demands attention. Predict, be surprised, respond

The recipe

- Predict the observation and its variance.
- Measure the innovation - the surprise.
- Update by the gain, or flag if too large.

Why it's powerful

- The same number drives learning and alerting.
- The gain adapts to how noisy things are.
- Nothing moves without a surprise.

Reproduce the Iteration, Then Scale Up

Direct Claude Code to check your arithmetic

GOAL Verify the predict-update step and the innovation monitor, then run both on real data.

[INPUT] Prompt · Claude Code

Write and run `part1_practice.py` that:

CASE A - one iteration:

- * from $x=100.0$, $P=0.20$, $Q=0.05$, $R=0.30$, $y=100.8$,
print the predict, innovation, S , gain, and update;

CASE B - innovation monitor:

- * run four steps on $[100.8, 100.5, 103.2, 100.9]$;
- * print nu , S , and $z = nu/\sqrt{S}$ each step;
- * flag any $|z| > 2$.

Then run the filter on a real price series and plot the standardized innovations with ± 2 bands.

Match every printed number to the slides.

Check against the slides

- predict $P_- = 0.25$.
- innovation 0.80 , $S = 0.55$.
- gain 0.455 , update 100.36 .
- step 3: $z = +4.08$ (alarm).

Deliverable: `part1_practice.py` whose printed numbers match the worked iteration and the z run.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What is the innovation?

**Observed minus
predicted - the
surprise**

Q2 What does the gain do to the surprise?

**Scales how much of
it corrects the state**

Q3 What does a large standardized innovation mean?

**News, a data error, or
a broken model**

All three come from the predict-update iteration and the standardized-innovation monitor.

Three Pitfalls to Avoid

Where predict-update slips

Common mistakes

- "Forget to grow the variance in predict."
- "Compare raw innovations across time."
- "Treat every big surprise as a trade."

The reality

- Predict adds Q - the filter loses certainty over time.
- Standardize by $\sqrt{S}$ before comparing surprises.
- A big z may be news or error - check before acting.

Two Calculations to Remember

Part 1 applications in one screen

1

One predict-update, by hand

Predict grew the variance to 0.25; the print's innovation was 0.80 with variance 0.55; the gain 0.455 moved the estimate to 100.36 and shrank the variance to 0.14. The surprise, scaled by the gain, is the whole update.

2

The standardized innovation is a signal

Dividing the innovation by $\sqrt{S}$ gives a z-score for surprise. Three steps sat inside ± 2 ; step 3's print of 103.2 spiked to $z = +4.08$ - a flag for news, error, or a broken model.

3

Nothing moves without a surprise

The innovation drives the update and doubles as an alarm. Predict, measure the surprise, and respond - by correcting the state, or by raising a flag.

Next — Part 2 practice: the Kalman gain itself, and how it balances trust between model and data.

★ WEEK 9 · PART 2 of 3

The Kalman Gain and the Full Recursion

The Number That Balances Model Against Data

The gain formula · the covariance P · the five equations · optimality

The update step corrects by the gain times the surprise. Part 2 derives that gain, shows how it tracks the filter's own uncertainty, assembles the full recursion, and states exactly when it is optimal.

2

PART 2 • THE GAIN

The Heart of the Filter

The Kalman gain and what it balances, the covariance that drives it, the full five-equation recursion assembled in one place, and the conditions under which the filter is optimal.

We Have the Cycle — Now, the Gain

Part 1 gave the two steps; Part 2 gives the weight

1

The correction is gain times surprise

Part 1 showed the update corrects the state by K times the innovation. But we left the gain K as a symbol — the one piece still missing.

2

The gain is the whole balancing act

It decides how much to trust the new data versus the model's prediction. Get it right and the estimate is optimal; get it wrong and you over- or under-react.

3

It falls out of the uncertainties

Remarkably, the optimal gain is determined entirely by the covariances the filter already tracks. No tuning — the math hands it to you.

Part 2 in one line: the gain optimally weighs model against data — and the filter computes it for free.

The Gain: The Heart of the Filter

How much to trust the new observation

Every correction the filter makes is scaled by the Kalman gain. It is a single number (or matrix) that answers, at every step: given what I know and how noisy the data is, how far should I move toward this observation?

What it controls

- The size of the correction in the update step.
- The balance between the model and the new data.
- How quickly the estimate reacts to change.

Why it is the heart

- Everything else is bookkeeping; the gain is the decision.
- It is where the filter's intelligence lives.
- The optimal gain is what makes the Kalman filter Kalman.

The Kalman Gain

State uncertainty over total uncertainty

$$K_t = P_{t|t-1} Z' F_t^{-1}, \quad F_t = Z P_{t|t-1} Z' + H$$

Reading it

- The numerator is the predicted state uncertainty.
- F , the denominator, is the innovation variance.
- F combines state uncertainty and measurement noise H .

The message

- The gain is a RATIO of uncertainties.
- More state uncertainty $\rightarrow$ larger gain $\rightarrow$ trust the data.
- More measurement noise $\rightarrow$ smaller gain $\rightarrow$ trust the model.

The Gain in One Dimension

A number between zero and one

$$\text{scalar case: } K = \frac{P_{\text{pred}}}{P_{\text{pred}} + H} \in [0, 1] \quad (\text{state unc.})/(\text{total unc.})$$

What it means

- In the scalar case, the gain lies between 0 and 1.
- It is the fraction of the surprise you keep.
- A gain of 0.4 moves the estimate 40% of the way to the data.

The extremes

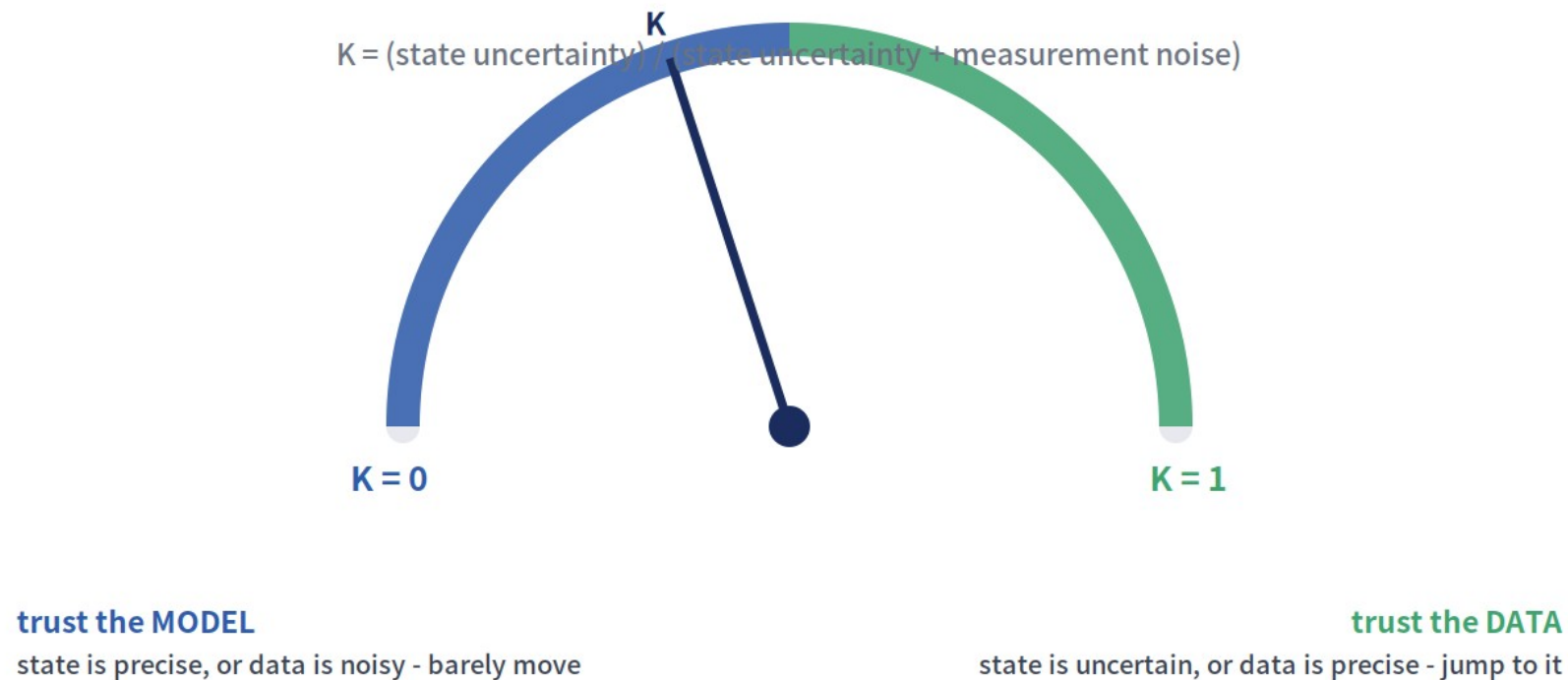
- Near 1: the data dominates — you nearly adopt the observation.
- Near 0: the model dominates — you barely budge.
- The filter picks the point that minimizes error.

The Gain as a Trust Dial

The gain is a dial from 0 to 1: trust the model when the state is precise or the data is noisy; trust the data when the state is uncertain or the measurement is sharp.

The Gain as a Trust Dial

The Kalman gain K is a number between 0 and 1 - how far to move toward the new data.



Trust the Model or Trust the Data?

The gain answers this at every step

The gain automatically resolves the central tension of estimation. When the model is more reliable than the data, it leans on the model; when the data is sharper, it leans on the data. No manual tuning.

When the model wins

- The state estimate is already precise (small P).
- Or the measurement is very noisy (large H).
- The gain is small — the observation barely moves you.

When the data wins

- The state estimate is uncertain (large P).
- Or the measurement is precise (small H).
- The gain is large — you jump toward the observation.

The Extremes of the Gain

Two behaviors at the ends of the dial

$K \rightarrow 1$: trust the DATA

$K \rightarrow 0$: trust the MODEL

Gain near one

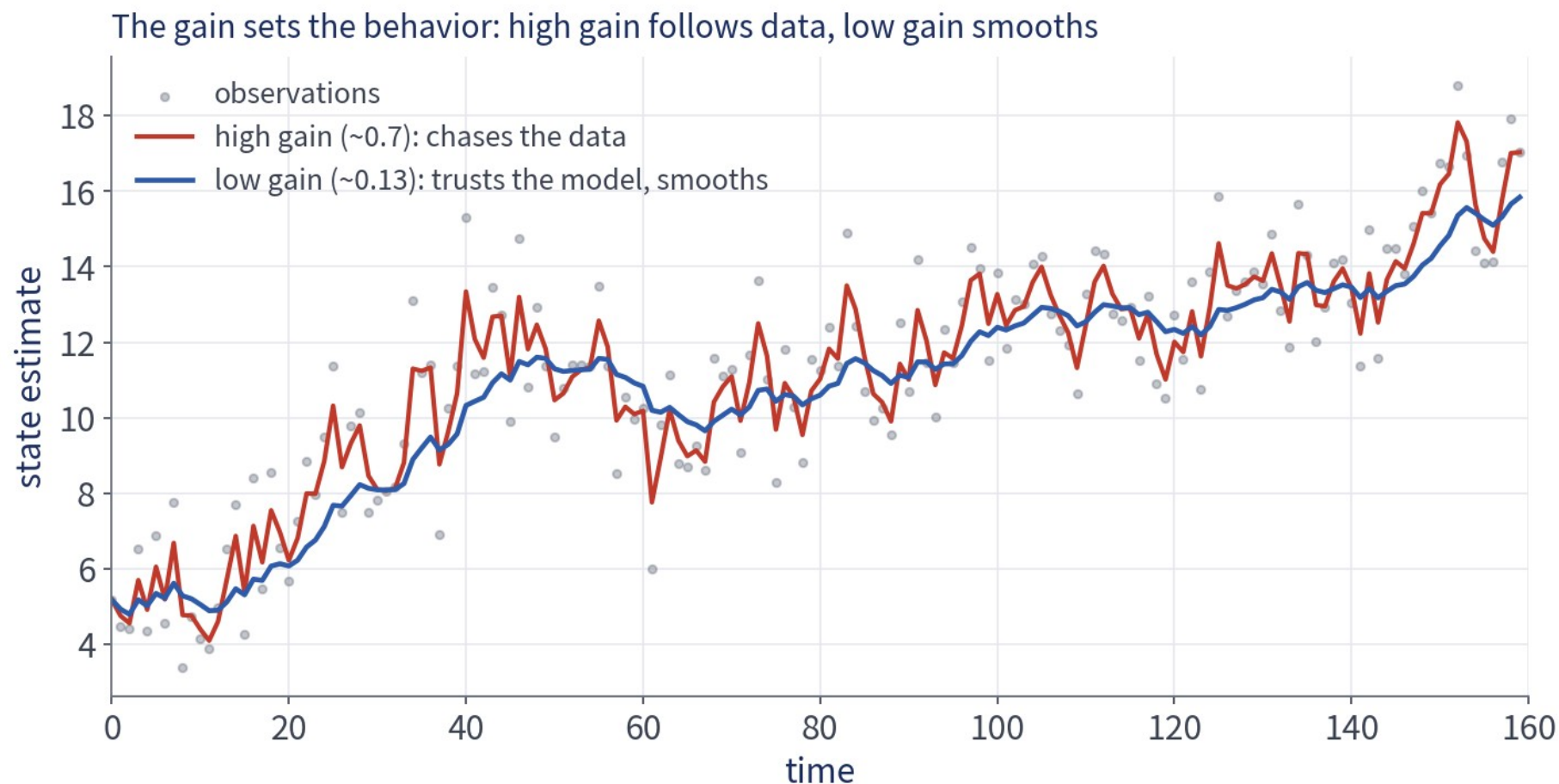
- The filter essentially adopts each new observation.
- It tracks fast but inherits the data's noise.
- Useful when the state truly moves quickly.

Gain near zero

- The filter mostly ignores the data and trusts the model.
- It is very smooth but slow to react.
- Useful when the state is nearly constant.

The Gain, in Action

Two filters on the same data: a high gain chases every observation (noisy but responsive); a low gain trusts the model and smooths (stable but slower). The gain sets the whole character.



The Covariance P

The filter knows how sure it is

covariance update: $P_{t|t} = (I - K_t Z) P_{t|t-1}$ (always shrinks)

What P is

- The covariance of the state estimate — its uncertainty.
- It grows in predict (by Q), shrinks in update.
- The update always reduces it: data adds information.

Why it matters

- P feeds directly into the gain each step.
- It gives calibrated error bars on every estimate.
- The filter literally tracks its own confidence.

Why the Update Shrinks P

Information can only help

The logic

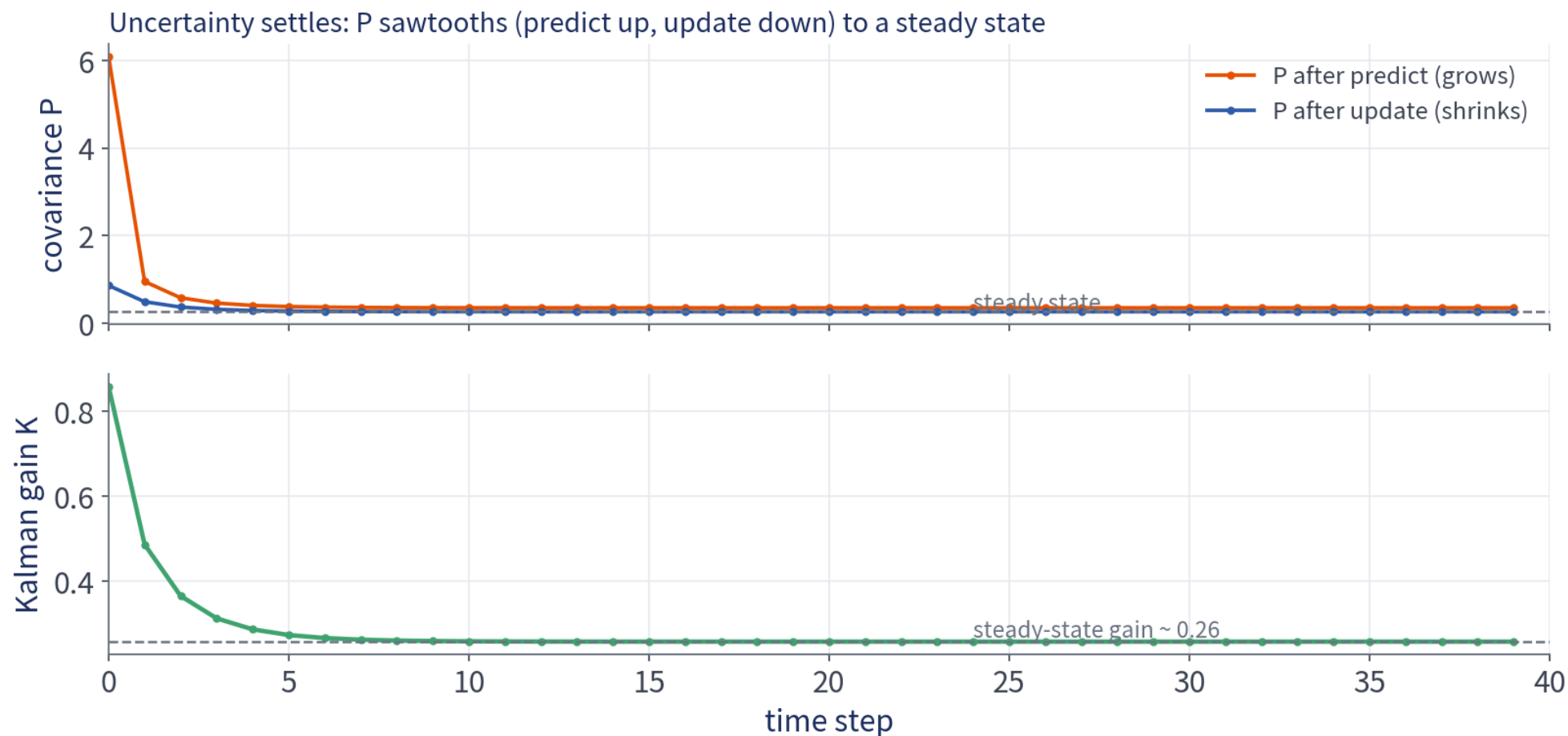
- An observation adds information about the state.
- More information means less uncertainty.
- So the update can only reduce P , never increase it.

The balance with predict

- Predict adds process noise Q — uncertainty grows.
- Update removes uncertainty — it shrinks.
- The two forces reach a balance over time.

Uncertainty and Gain Settle

The covariance sawtooths — up in predict, down in update — and converges (top). The gain converges with it to a steady-state value (bottom). A time-invariant filter reaches equilibrium.



The Gain Adapts as P Changes

High early, then settling

Early on

- The initial state is uncertain, so P is large.
- That makes the gain high — the filter trusts data heavily.
- It converges quickly toward the truth.

Later

- As information accumulates, P shrinks and stabilizes.
- The gain settles to a steady value.
- The filter reaches a stable trust between model and data.

The Full Recursion at a Glance

Five equations, two steps — assembled

$$\begin{array}{ll} \text{PREDICT} & \hat{x}_{t|t-1} = T \hat{x}_{t-1|t-1} \quad P_{t|t-1} = T P_{t-1|t-1} T' + Q \\ \text{innovation} & v_t = y_t - Z \hat{x}_{t|t-1} \quad F_t = Z P_{t|t-1} Z' + H \\ & \text{gain} \\ \text{UPDATE} & \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t \quad P_{t|t} = (I - K_t Z) P_{t|t-1} \end{array}$$

The two steps

- Predict: move the state and grow P.
- Update: gain, then correct the state and shrink P.
- The innovation v and its variance F link the two.

How to use it

- Memorize the shape, not the matrix algebra.
- Predict has no data; update is where data enters via K .
- Everything reduces to these five lines.

The Five Equations, Named

Each line, and its job

Step	Equation	What it does
Predict state	$x = T x$	move the state forward
Predict cov	$P = T P T' + Q$	grow the uncertainty
Gain	$K = P Z' / F$	weigh model vs data
Update state	$x = x + K v$	correct by gain times surprise
Update cov	$P = (I - K Z) P$	shrink the uncertainty

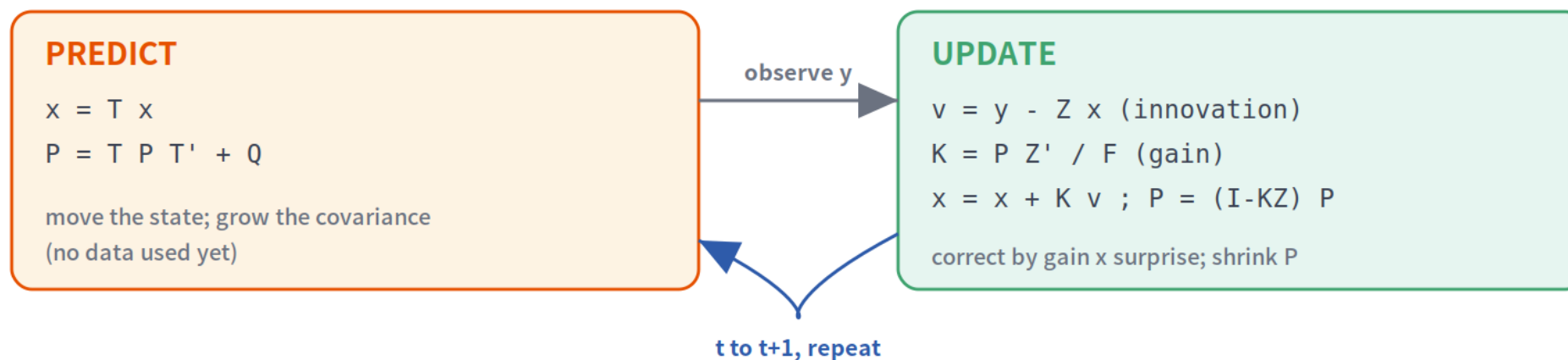
$F = Z P Z' + H$ is the innovation variance; $v = y - Z x$ is the surprise. That is the entire filter.

The Full Recursion, in the Loop

The five equations placed in the predict-update loop. Two carried objects — the state x and covariance P — summarize the entire past; the gain K is the only place data enters.

The Full Recursion, in the Loop

Five equations, two steps - each observation drives one full pass.



Two objects carry the whole past

The state estimate x and its covariance P are all you carry - a complete, compact summary.

Predict inflates P by Q ; update deflates it via the gain. The gain K is the only place data enters.

$F = Z P Z' + H$ is the innovation variance - the denominator that scales the gain.

When Is the Kalman Filter Optimal?

Powerful, but know the fine print

linear + Gaussian $\Rightarrow$ Kalman filter = optimal (minimum-variance) estimator

The conditions

- The system is linear (the two equations are linear).
- The noises are Gaussian.
- The matrices T , Z , Q , H are known.

What 'optimal' means

- It is the minimum-variance estimator of the state.
- It equals the exact posterior mean — nothing does better.
- Under these conditions, the Kalman filter is unbeatable.

Optimality: The Fine Print

When the assumptions bend

When they fail

- Nonlinear dynamics — the plain filter no longer applies.
- Non-Gaussian, heavy-tailed noise — optimality is lost.
- Unknown parameters must be estimated first.

The fixes

- Extended or unscented Kalman filters for nonlinearity.
- Robust or particle filters for non-Gaussian noise.
- Estimate T , Z , Q , H by maximum likelihood, then filter.

The Steady-State Gain

When the gain stops changing

The idea

- For a time-invariant system, P and K converge.
- After convergence, the gain is a fixed constant.
- You can precompute it and skip updating P each step.

Why it is useful

- It is cheaper — no covariance recursion at run time.
- It reveals the filter's long-run behavior at a glance.
- Many simple exponential smoothers are steady-state filters.

Filtering vs Smoothing, Revisited

The same filter, two ways to use it

The Kalman filter is a forward pass — it gives filtered (real-time) estimates. Add a backward pass over the same quantities and you get the smoother: the best estimate of each past state using all data.

Filtering

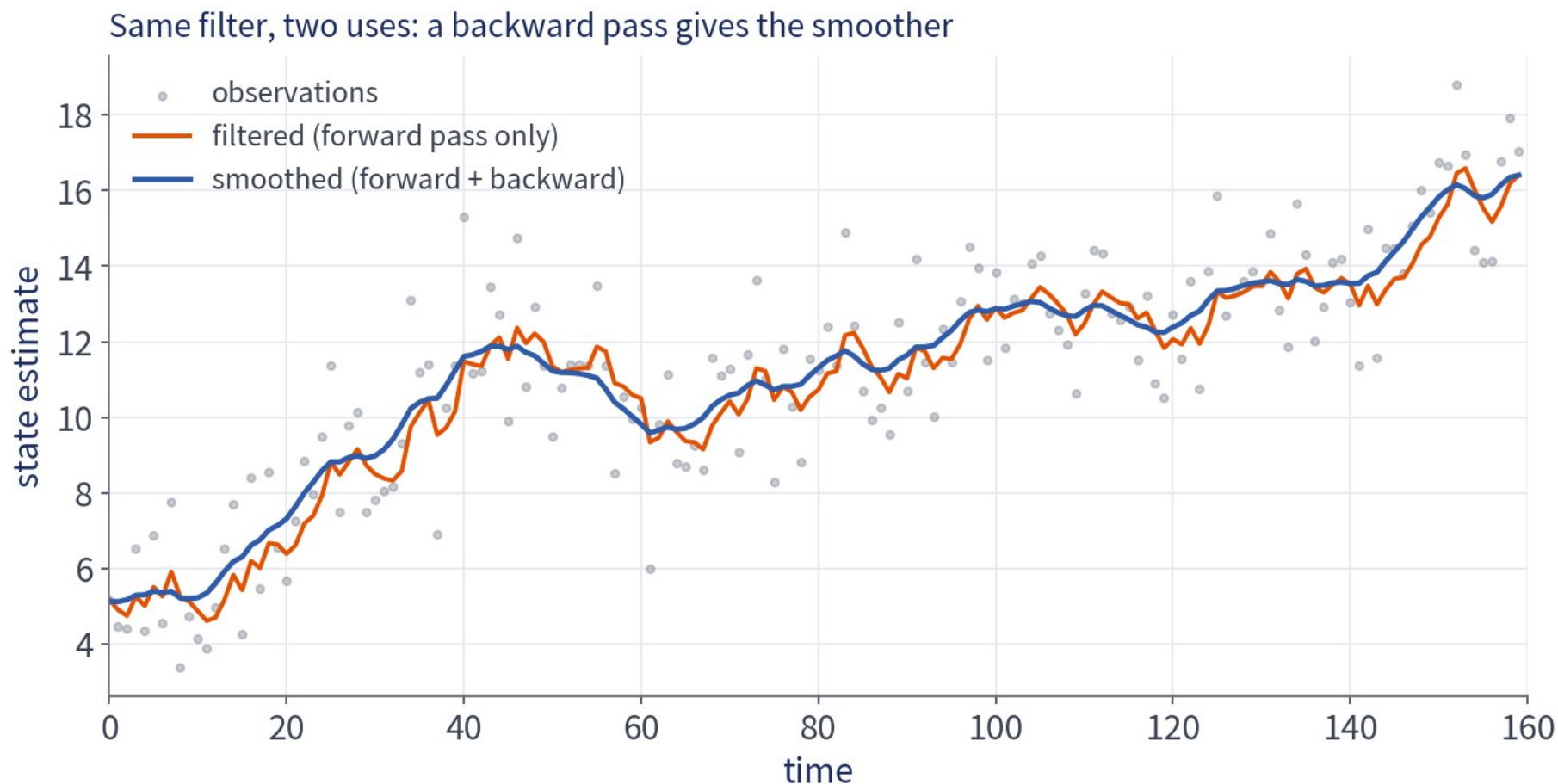
- One forward pass, causal, real-time.
- Uses data up to each point only.
- The output you trade on.

Smoothing

- A backward pass added on top of the filter.
- Uses the whole sample for every past point.
- Cleaner, but only available after the fact.

One Filter, Two Uses

The forward pass gives the filtered estimate (real-time). A backward pass over the same stored quantities produces the smoother — cleaner, using all data, but only in hindsight.



Watch the Gain and Covariance

Direct Claude Code to expose the filter's internals

GOAL Instrument a Kalman filter to see the gain and covariance evolve, and confirm they converge.

[INPUT] Prompt · Claude Code

Write and run `gain_covariance_lab.py` that:

1. runs a local level Kalman filter and records the covariance P and the gain K at every step;
 2. plots P (predicted and updated) and K over time, showing them converge to steady state;
 3. re-runs with a LARGE and a SMALL process noise Q , and compares the steady-state gains;
 4. verifies the update step always reduces P .
- Explain how a bigger Q leads to a bigger steady-state gain (trusting the data more).

How to work with it

- P should sawtooth and settle.
- K should converge to a constant.
- Bigger $Q \rightarrow$ bigger gain $\rightarrow$ faster tracking.
- Update must never increase P .

Deliverable: `gain_covariance_lab.py` + convergence plots and a Q comparison.

Three Questions Before You Move On

If you can answer these, Part 2 has landed

Q1	The gain is a ratio of what to what?	State unc. / total unc.
Q2	Measurement noise H rises sharply. Which way does the gain move?	Down — trust the model
Q3	What two conditions make the Kalman filter optimal?	Linear + Gaussian

All three come from the gain formula, the trust dial, and the optimality conditions.

Three Misconceptions to Avoid

Where gain intuition slips

Common myths

- "You tune the gain by hand."
- "A higher gain is always better — it reacts faster."
- "The Kalman filter is always optimal."

The reality

- The optimal gain is computed from the covariances — no tuning.
- High gain reacts fast but inherits noise; there is a trade-off.
- Optimality needs linearity, Gaussian noise, and known parameters.

Three Things to Carry Forward

Part 2 in one screen

1

The gain balances model against data

It is a ratio of uncertainties: large when the state is uncertain or the data is sharp, small otherwise. It is computed, not tuned — the filter hands it to you.

2

The covariance drives everything

P grows in predict, shrinks in update, and feeds the gain each step. It gives calibrated error bars and, for time-invariant systems, converges to a steady state.

3

Five equations, optimal under clear conditions

Predict two, gain one, update two — the whole filter. It is the minimum-variance estimator when the system is linear and the noise is Gaussian.

Next — Part 3: Kalman in practice — denoising, a dynamic beta, and pairs trading Stage III.

★ WEEK 9 · PART 2 · PRACTICE

The Kalman Gain, Worked by Hand

Turn the Dial Between Model and Data; Tune the Filter

How data quality and the Q/R ratio set the gain

Part 2's dial, with the arithmetic shown. We compute the gain across three data-quality regimes, watch the same surprise move the estimate differently, then find the steady-state gain and tune a filter from smooth to responsive.

2P

PART 2 · APPLICATIONS

The Dial Between Trust and Doubt

Case A: compute the gain for a clean quote, a normal print, and a noisy one - and watch the same surprise land differently. Case B: find the steady-state gain and tune a filter slow or fast with Q/R . Then a Claude Code lab.

What Part 2 Told Us

Now we turn the dial - with numbers

1

The gain balances model and data

K is state uncertainty over total uncertainty - a number between zero and one that says how much to trust the new print.

2

It shrinks the uncertainty

Every update lowers the variance P , so the filter grows more confident - and the gain adapts as P changes.

3

It settles to a steady state

After a while the gain stops changing, at a level set by the Q/R ratio. We compute that steady gain by hand.

Everything here is a calculator exercise: three noise levels, one steady-state formula, two tunings.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from the gain formula.
- Compute the gain for a few settings.
- Then read what it does to the estimate.

What you should watch

- How cleaner data raises the gain.
- How the same surprise lands differently.
- How Q/R tunes the filter's responsiveness.

The Gain Is a Dial

Between trusting the model and the data

The Kalman gain answers one question at every step: trust the model, or trust the new print? The answer depends on how noisy the data is. We fix the state uncertainty and compute the gain for a clean exchange quote, a normal print, and a noisy thin one - then watch the same surprise move the estimate by very different

The setup

- State uncertainty $P_- = 0.25$, fixed.
- The same innovation, 0.8, each time.
- Only the data noise R changes.

What we'll do by hand

- Compute the gain for three noise levels.
- Turn each gain into a correction.
- See clean data move the estimate most.

The Gain Formula

State uncertainty over total uncertainty

$$K = \frac{P^-}{P^- + R} = \frac{\text{state uncertainty}}{\text{state uncertainty} + \text{data noise}} \quad (\text{a number between 0 and 1})$$

Reading it

- The numerator is how unsure the model is.
- The denominator adds the data noise.
- So K is always between 0 and 1.

The two limits

- Tiny data noise: K near 1, trust the print.
- Huge data noise: K near 0, trust the model.
- Everything else sits on the dial between.

Three Data-Quality Regimes

Same state, different data noise

clean quote ($R = 0.05$) : $K = \frac{0.25}{0.30} = \mathbf{0.83}$ (trust the data)

normal ($R = 0.25$) : $K = \frac{0.25}{0.50} = 0.50$

noisy print ($R = 1.00$) : $K = \frac{0.25}{1.25} = \mathbf{0.20}$ (trust the model)

A clean quote ($R=0.05$) gives a gain of 0.83 - trust the data; a noisy print ($R=1.0$) gives 0.20 - trust the model. Same filter, opposite behavior.

Same Surprise, Different Correction

The gain scales the innovation

$$\text{correction} = K \times \nu : \quad 0.83(0.8) = 0.67, \quad 0.50(0.8) = 0.40, \quad 0.20(0.8) = 0.16$$

What happened

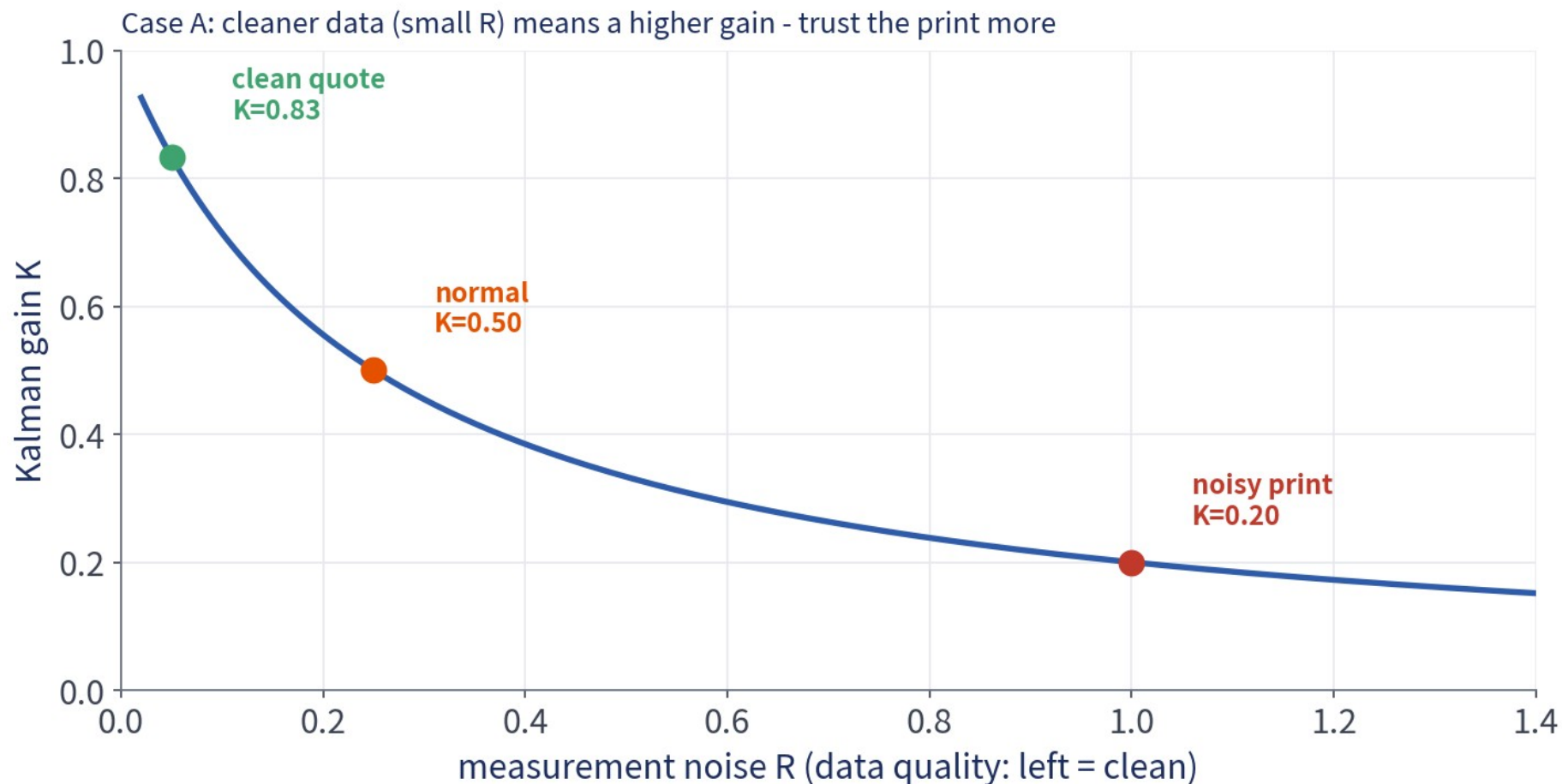
- The innovation was 0.8 in every case.
- The clean quote moved the estimate 0.67.
- The noisy print moved it only 0.16.

The lesson

- The surprise is the same size each time.
- The gain decides how much of it to believe.
- Data quality, not the surprise, sets the response.

The Gain Dial

As the data gets noisier (R grows to the right), the gain falls: the filter trusts each print less and leans on the model more. The clean quote, the normal print, and the noisy print sit at three points along the same smooth dial



Why This Is Automatic

The filter reads data quality for you

Without the gain

- You'd pick one blend of model and data.
- It would be wrong when data quality changes.
- A thin-market print would fool you.

With the gain

- Noisy prints are down-weighted automatically.
- Clean quotes get taken more seriously.
- The dial turns itself, every step.

Tuning the Filter

From smooth and laggy to fast and jumpy

Run the filter a while and the gain stops changing - it reaches a steady state set by the ratio of process noise Q to measurement noise R . That ratio is your tuning knob: a small Q gives a smooth, laggy filter; a large Q gives a responsive, noisier one. We compute the steady-state gain for both by hand

The idea

- The variance settles, so the gain settles.
- The steady gain depends only on Q and R .
- Q/R is the responsiveness knob.

What we'll do by hand

- Use the steady-state gain formula.
- Compute it for a slow and a fast filter.
- See the trade-off between smooth and quick.

Why the Gain Settles

Predict grows P , update shrinks it

The tug of war

- Predict adds Q - uncertainty grows.
- Update multiplies by $(1-K)$ - it shrinks.
- The two forces reach a balance.

At the balance

- The variance stops changing: steady state.
- So the gain stops changing too.
- It depends only on Q and R , not the data.

The Steady-State Gain

When the dial stops turning

$$p = \frac{Q + \sqrt{Q^2 + 4QR}}{2}, \quad K_{\infty} = \frac{p}{p+R} \quad (\text{the gain stops changing})$$

Solve the balance for the steady predicted variance p , then form the gain. It no longer depends on the data - only on how fast the state drifts versus how noisy the prints are.

Slow vs Fast

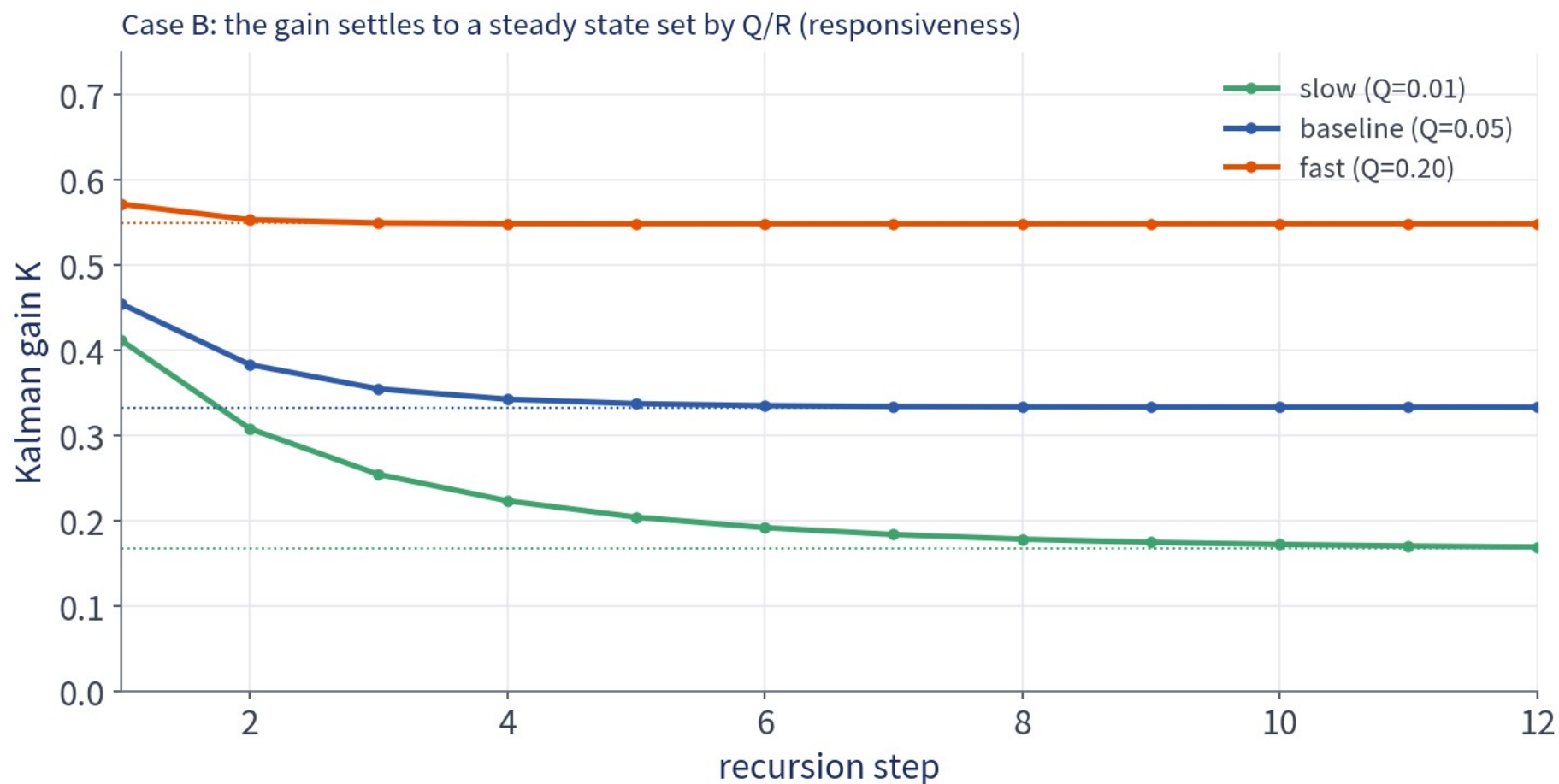
Two tunings, two steady gains

$$\begin{aligned} \text{slow } (Q = 0.01) : p = 0.060, \quad K_{\infty} = \frac{0.060}{0.360} = \mathbf{0.17} \quad (\text{smooth, laggy}) \\ \text{fast } (Q = 0.20) : p = 0.365, \quad K_{\infty} = \frac{0.365}{0.665} = \mathbf{0.55} \quad (\text{responsive, noisy}) \end{aligned}$$

A small Q settles to a gain of 0.17 - smooth but slow to react; a large Q settles to 0.55 - quick to follow moves but noisier. You pick the trade-off.

The Gain Settles to Steady State

From the same start, each filter's gain converges to its own steady level: the slow filter to 0.17, the baseline to 0.33, the fast one to 0.55. The Q/R ratio, chosen in advance, decides where the dial finally rests.



Responsive or Smooth?

A design choice you control

A fast filter (high Q)

- Catches regime shifts quickly.
- But passes more noise into the estimate.
- Good when the state really moves.

A slow filter (low Q)

- Gives a smooth, stable estimate.
- But lags when the state actually changes.
- Good when the state is nearly constant.

The Common Pattern

The gain is the whole story

Both cases are about one number. Moment to moment, the gain reads data quality and decides how much of each surprise to believe. Over the long run, the Q/R ratio you chose fixes where that dial settles. Master the gain, and you have mastered the filter's judgment

Short run

- The gain rises with cleaner data.
- It scales each surprise into a correction.
- It turns itself, step by step.

Long run

- The variance and gain settle to steady state.
- Q/R sets responsiveness versus smoothness.
- You tune it before you ever see the data.

Reproduce the Dial and the Tuning

Direct Claude Code to check your arithmetic

GOAL Verify the gain across regimes and the steady-state tuning, then plot both.

[INPUT] Prompt · Claude Code

```
Write and run part2_practice.py that:  
CASE A - the dial:  
  * with  $P_- = 0.25$ , print  $K = P_- / (P_- + R)$  for  
     $R = 0.05, 0.25, 1.0$ ; print  $K * 0.8$  each time;  
CASE B - steady state:  
  * with  $R = 0.30$ , compute  $p = (Q + \sqrt{Q^2 + 4QR}) / 2$   
    and  $K = p / (p + R)$  for  $Q = 0.01, 0.05, 0.20$ ;  
  * run the recursion and confirm  $K$  converges there.  
Then plot  $K$  vs  $R$  (the dial) and  $K$  vs step (settling).  
Match every printed number to the slides.
```

Check against the slides

- $K = 0.83, 0.50, 0.20$.
- corrections $0.67, 0.40, 0.16$.
- steady $K = 0.17, 0.33, 0.55$.
- gain converges to those.

Deliverable: part2_practice.py whose printed numbers match the dial and the steady-state tunings.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What happens to the gain as data gets noisier?

It falls - trust the model more

Q2 What sets the steady-state gain?

The Q/R ratio, not the data

Q3 What does a large Q give you?

A fast, responsive, noisier filter

All three come from the gain dial and the steady-state tuning.

Three Pitfalls to Avoid

Where the gain trips people up

Common mistakes

- "A high gain is always better."
- "Set Q and R once and forget the ratio."
- "The steady gain depends on the data."

The reality

- A high gain is jumpy - it passes noise through.
- Q/R is a design choice: smooth vs responsive.
- The steady gain depends only on Q and R.

Two Calculations to Remember

Part 2 applications in one screen

1

The gain is a dial on data quality

With state uncertainty fixed at 0.25, a clean quote ($R=0.05$) gave a gain of 0.83 and a noisy print ($R=1.0$) gave 0.20 - so the same 0.8 surprise moved the estimate 0.67 versus 0.16. The filter reads data quality for you.

2

The gain settles, and Q/R tunes it

The steady-state gain solves a simple balance: a slow filter ($Q=0.01$) settled to 0.17, smooth but laggy; a fast one ($Q=0.20$) to 0.55, responsive but noisy. You choose the trade-off with Q/R.

3

Master the gain, master the filter

Short run, the gain weighs each surprise; long run, Q/R fixes where the dial rests. Everything the Kalman filter does flows through this one number.

Next — Part 3 practice: the pairs trade and nowcasting, where the filter meets real markets.

★ WEEK 9 · PART 3 of 3

The Kalman Filter in Practice

Two Flagship Applications: Pairs Trading and Nowcasting

A self-adjusting hedge ratio · a real-time read on the economy

With the recursion and the gain in hand, we put the filter to work on two landmark problems: an adaptive hedge ratio for pairs trading, and nowcasting — estimating the economy's state right now from a stream of noisy, mixed-frequency data.

3

PART 3 · IN PRACTICE

Two Faces of One Filter

A dynamic hedge ratio that keeps a pairs trade market-neutral, and nowcasting that reads current GDP in real time from a ragged stream of indicators — both the same predict-update engine.

From the Recursion to Real Use

Parts 1-2 built it; Part 3 applies it twice

1

We have the full filter

Predict, update, the gain, and the covariance. The machinery is complete — now we use it on two very different problems.

2

Example 1 — pairs trading

Let the hedge ratio itself drift as a hidden state, so the spread stays market-neutral as the relationship evolves.

3

Example 2 — nowcasting

Read the current state of the economy in real time from many noisy, mixed-frequency indicators that arrive on a ragged schedule.

Part 3 in one line: the same recursion powers an adaptive hedge and a real-time economic nowcast.

Two Applications, One Filter

Different fields, identical machinery

Pairs trading and nowcasting look unrelated — one is a trading strategy, the other a macro forecast. Yet both treat what they want as a hidden state seen through noisy data, and both use the very same Kalman recursion.

Example 1 — pairs trading

- The hidden state is a drifting hedge ratio.
- The observations are the two asset prices.
- Goal: keep the traded spread market-neutral.

Example 2 — nowcasting

- The hidden state is the current state of the economy.
- The observations are many mixed-frequency indicators.
- Goal: estimate current-quarter GDP, right now.

The Shared Capability: Denoising

Both extract a clean state from noise

What the filter always does

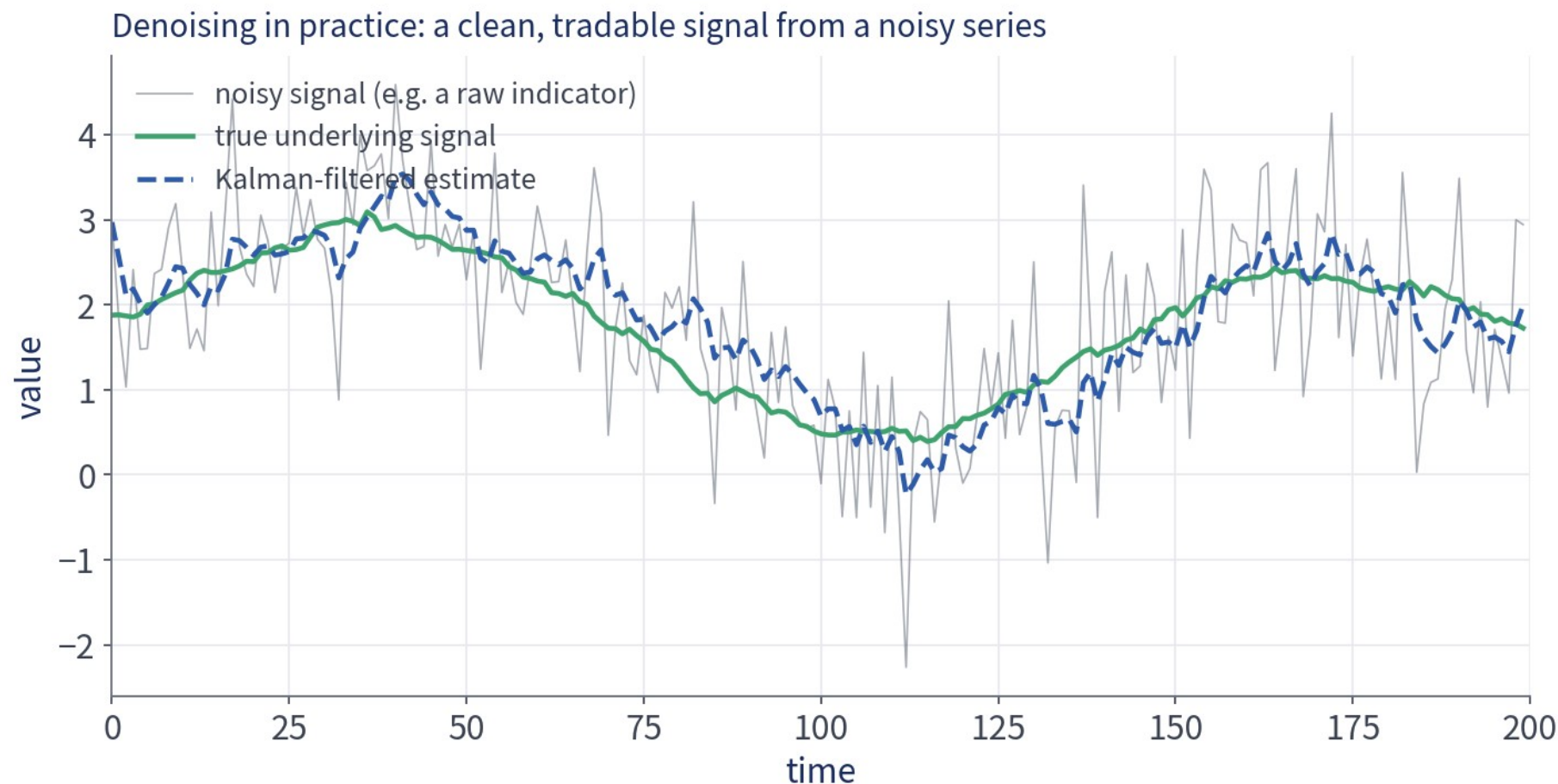
- Treats the quantity you want as a hidden state.
- Fuses a model with noisy data, weighted by uncertainty.
- Returns a clean, real-time estimate with error bars.

Why this generalizes

- A hedge ratio, a factor, a level — all just hidden states.
- The observation model changes; the recursion does not.
- Learn it once, apply it across finance and macro.

The Core Move: Signal From Noise

In both applications the filter does the same thing — a raw, noisy series (grey) becomes a clean estimate of the underlying hidden state (dashed), tracking the truth (green) in real time.



A Regression That Adapts

Letting the hedge ratio drift over time

Ordinary regression fixes a coefficient for all time. But the relationship between two assets drifts. The Kalman filter lets the hedge ratio itself be a hidden state that evolves — a regression that quietly updates.

The shift in view

- The hedge ratio is no longer one fixed number.
- It becomes a state with its own gentle dynamics.
- The filter re-estimates it at every observation.

Why it matters

- Market betas and exposures drift with the regime.
- A static estimate is stale the moment things move.
- A dynamic hedge ratio stays current, automatically.

The Dynamic Regression as State Space

Make the hedge ratio the state

dynamic regression: $Y_t = \beta_t X_t + \varepsilon_t$, $\beta_t = \beta_{t-1} + \eta_t$ (β is the state)

The two equations

- The observation: Y is beta-times- X plus noise.
- The state: beta follows a slow random walk.
- It is exactly a state space model — so Kalman applies.

The single knob

- How fast beta may drift is set by its process noise.
- Large: beta adapts quickly but jitters.
- Small: beta is stable but slow to move.

Why a Rolling Window Isn't Enough

The usual quick fix, and its flaws

The rolling-window fix

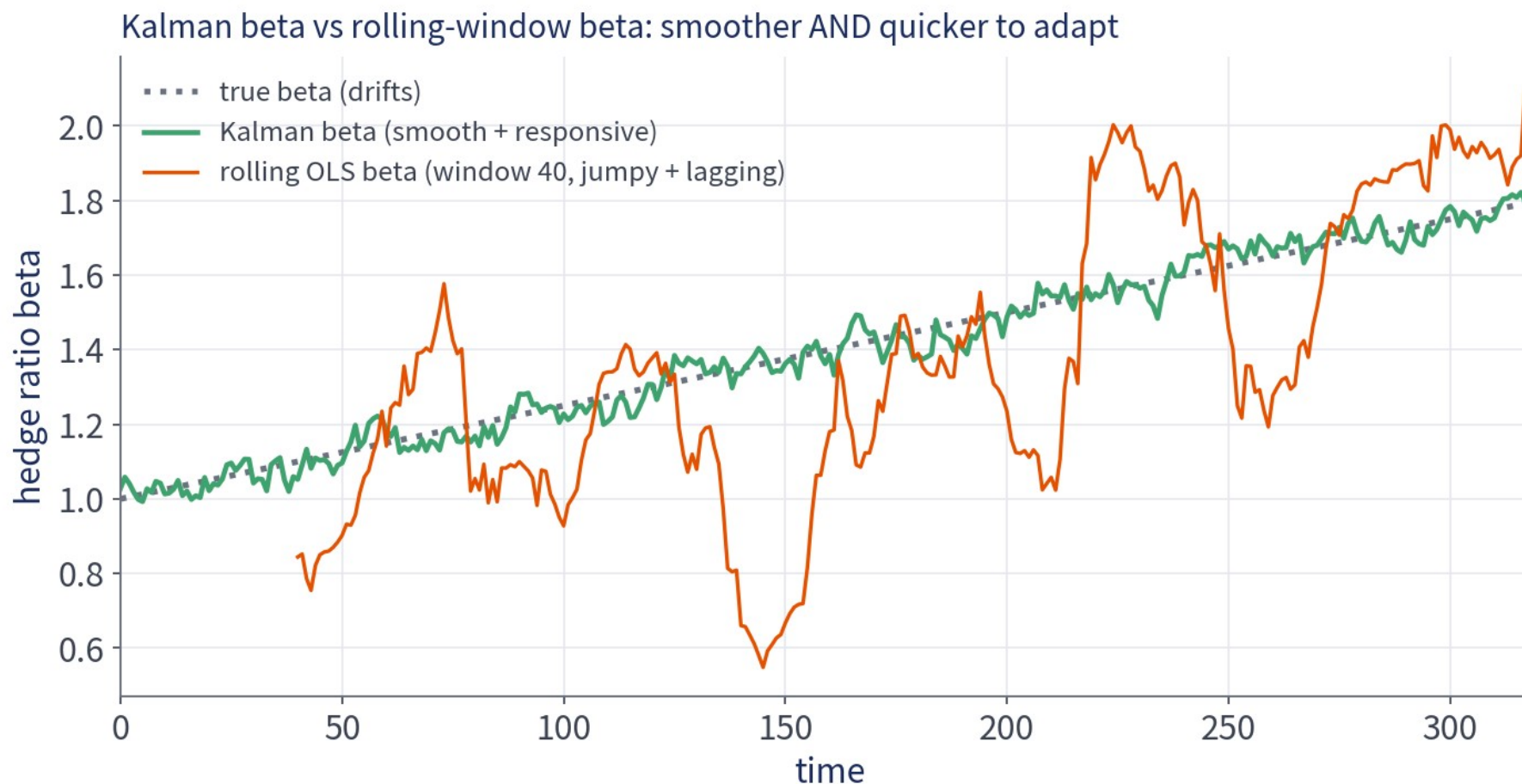
- Re-estimate the coefficient over the last N points.
- It does adapt — but only crudely.
- The window length is an arbitrary, awkward choice.

Why Kalman is better

- No window: it weights all history by relevance.
- It reacts faster AND stays smoother.
- It gives principled uncertainty, not just a point.

Kalman Beta vs Rolling-Window Beta

The true hedge ratio drifts (dotted). A rolling OLS beta (orange) is jumpy and lags; the Kalman beta (green) is both smoother and quicker to adapt — with no window length to guess.



Stage III: The Time-Varying Hedge Ratio

The thread gains a self-adjusting equilibrium

The pairs trade has had volatility-scaled bands (Stage I) and a cointegration anchor (Stage II). Stage III lets the hedge ratio itself drift — so the spread stays centered even as the relationship evolves.

The problem it solves

- A static hedge ratio slowly goes stale as beta drifts.
- The 'market-neutral' spread starts to leak exposure.
- Re-fitting by hand is clumsy and always behind.

The Stage III fix

- Model the hedge ratio as a Kalman state.
- It updates every day, keeping the spread centered.
- The trade stays neutral as the market changes.

The Adaptive Spread and Signal

A drifting hedge, a familiar trade

adaptive spread: $z_t = Y_t - \beta_t X_t \Rightarrow$ trade $\frac{z_t - \mu}{\sigma}$: enter $|\cdot| \geq 2$, exit ≈ 0

The spread

- Form the spread with the CURRENT filtered hedge ratio.
- Because beta adapts, the spread stays centered.
- It is a cleaner, more stable quantity to trade.

The signal

- Z-score the adaptive spread as before.
- Enter when it is stretched, exit as it reverts.
- The mechanics are Stage II's; the hedge is now live.

Why the Adaptive Spread Is Cleaner

A hedge that keeps up

Static hedge (Stage II)

- One beta fixed over the whole sample.
- As the true ratio drifts, the spread decenters.
- Signals degrade; false extremes appear.

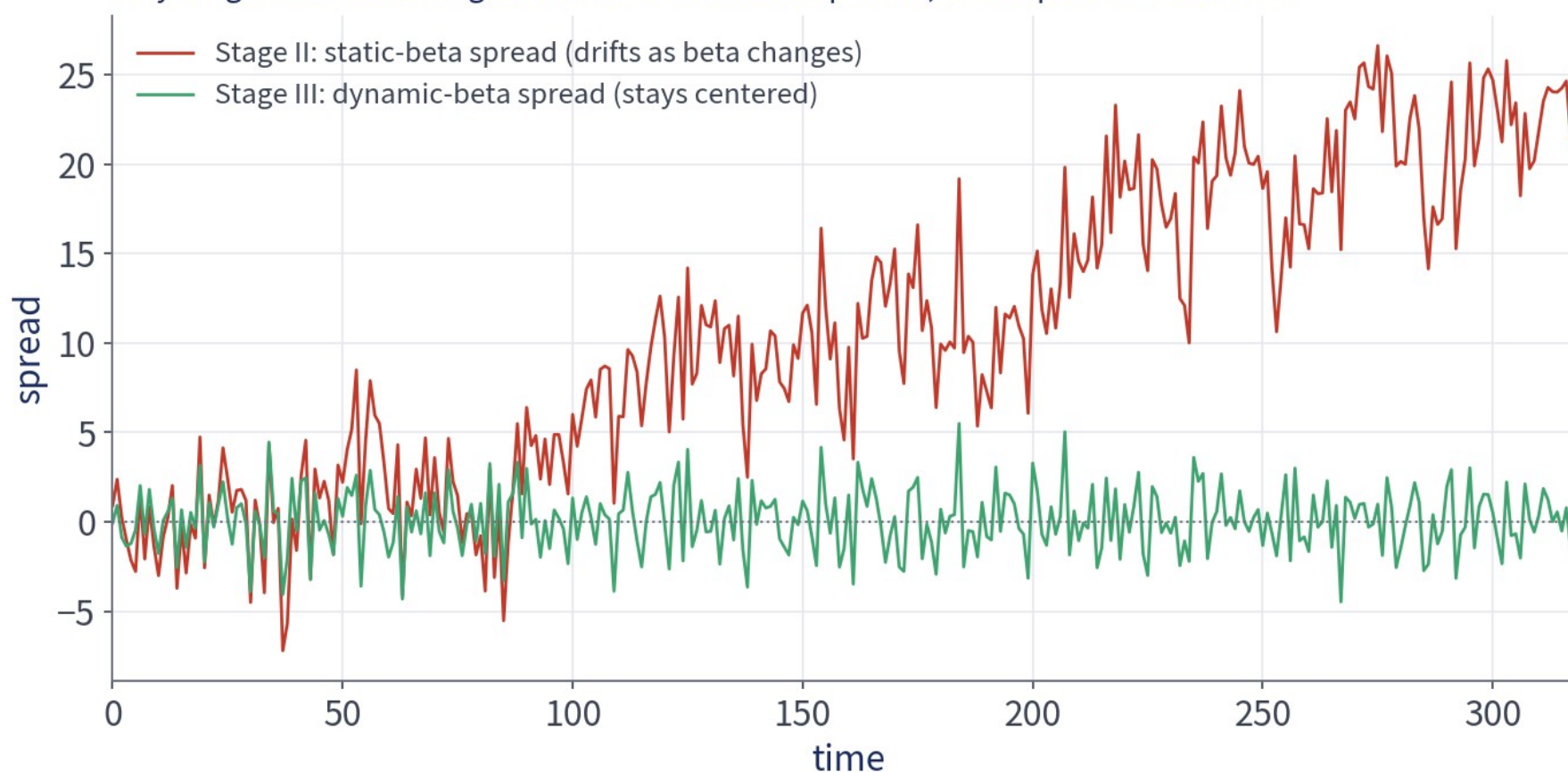
Dynamic hedge (Stage III)

- Beta tracks the true ratio in real time.
- The spread stays centered on zero.
- Signals stay meaningful as the regime shifts.

Static vs Adaptive Spread

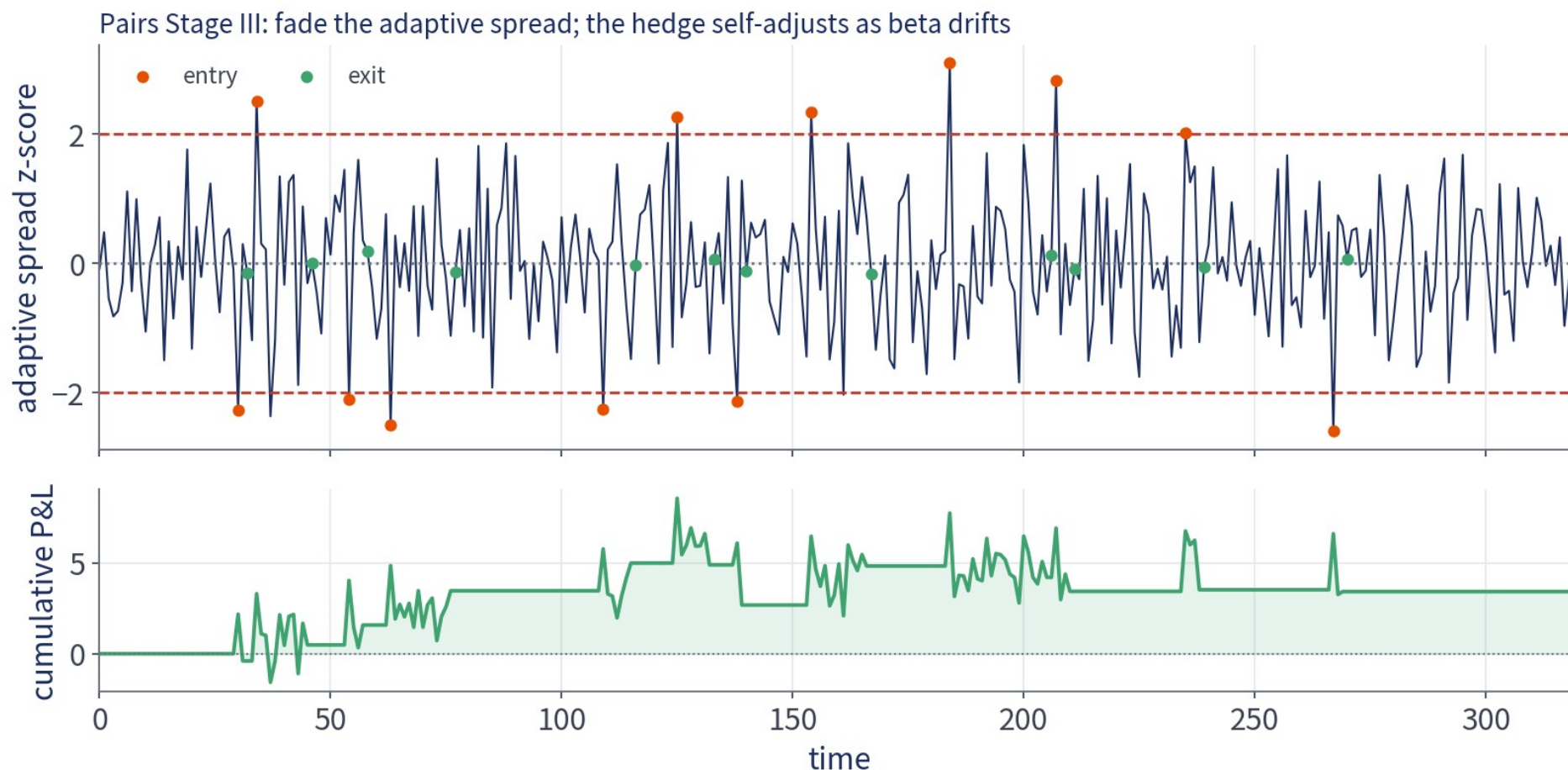
As the true hedge ratio drifts, the static-beta spread (red) leaks away from zero — producing false signals. The dynamic-beta spread (green) stays centered, keeping the trade honest.

Why Stage III: a static hedge leaks as the relationship drifts; an adaptive one does not



Pairs Stage III, in Action

Fade the adaptive spread's z-score at the bands and exit near zero. Because the hedge self-adjusts, the spread reverts cleanly and the cumulative P&L (below) accrues steadily.



Stage III vs Stage II

What the Kalman hedge adds

Aspect	Stage II (Week 7)	Stage III (Week 9)
Hedge ratio	static — one OLS beta	dynamic — a drifting state
Adapts to drift	no	yes, in real time
Spread when beta shifts	leaks off-center	stays centered
Underlying tool	cointegration + VECM	the Kalman filter

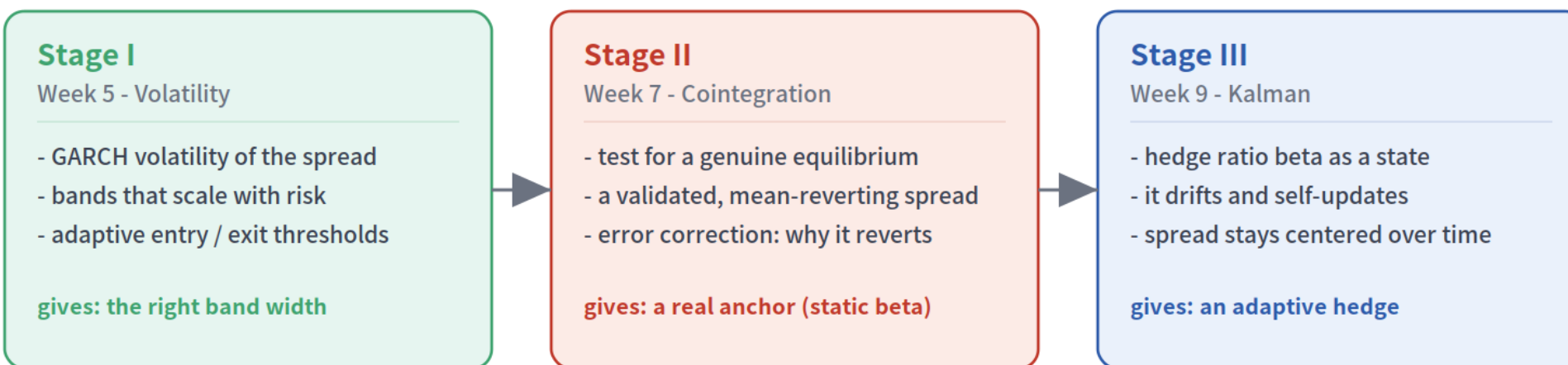
Stage III does not replace the cointegration anchor — it lets that equilibrium itself evolve.

The Complete Pairs Journey

Three stages across three weeks: volatility-scaled bands (W5), a cointegration anchor (W7), and a Kalman-adaptive hedge (W9). Each layer deepens the same market-neutral trade.

The Complete Pairs Journey

Three weeks, three stages - each adds a layer to the same trade.



The same idea, deepened each week

Find a pair, size the bands by volatility, anchor it with cointegration, and let the hedge ratio adapt.

Stage III does not replace II - it lets the equilibrium itself evolve, keeping the trade market-neutral.

What Is Nowcasting?

A real-time read on the economy

Official GDP arrives with a long lag and big revisions. Nowcasting estimates it RIGHT NOW — the present, not the future — by fusing the many timelier indicators that trickle out during the quarter. It is a landmark Kalman-filter application.

The idea

- Predict the present, before the official number exists.
- Use employment, surveys, production, sales — as they release.
- Introduced by Giannone, Reichlin and Small (2008).

Who uses it

- Central banks run live nowcasts of GDP.
- The New York and Atlanta Fed publish such models.
- It is now standard practice in policy and markets.

Mixed Frequency and the Ragged Edge

Why nowcasting is hard

Mixed frequency

- GDP is quarterly; most indicators are monthly or weekly.
- They must be combined despite different clocks.
- You need a model that speaks all frequencies at once.

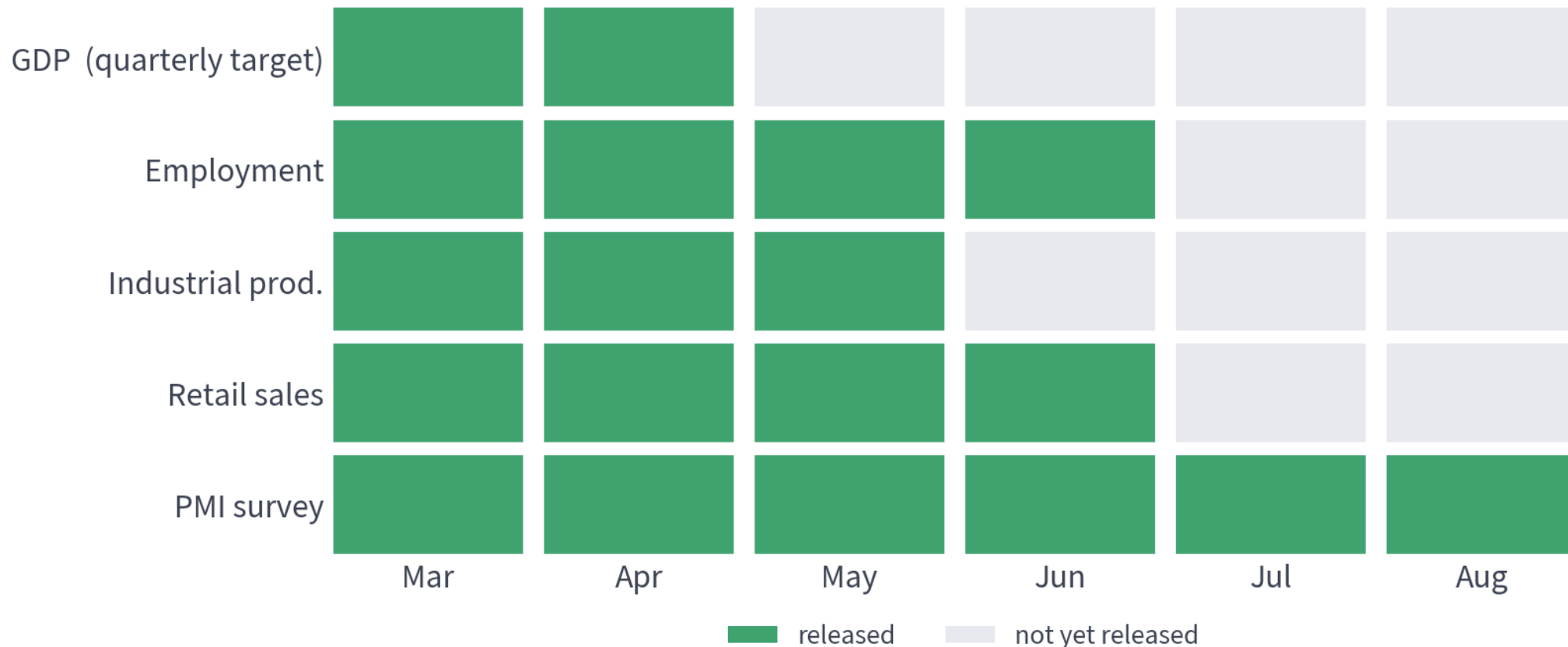
The ragged edge

- Indicators release on different schedules and delays.
- At any moment, each series ends at a different date.
- So the data panel has a jagged, incomplete edge.

The Ragged Edge

At any moment, series end at different dates: surveys are timely, hard data lag, GDP lags most. This jagged pattern of missing recent values is exactly what the Kalman filter handles gracefully.

The ragged edge: at any moment, series have different last-available dates



Why the Kalman Filter Is Natural Here

Missing data and real-time updating, built in

Two properties make the filter perfect for nowcasting: it handles missing data by simply skipping the update, and it updates in real time as each release arrives. The ragged edge that breaks other methods is a non-issue.

Missing data, handled

- If a series hasn't been released, skip its update step.
- The state still evolves; you just don't correct it there.
- The ragged edge needs no special patching.

Real-time, by design

- Each new release triggers one update — no full refit.
- The nowcast revises the instant data lands.
- Exactly the recursive advantage from Part 1.

The Dynamic Factor Model

One common factor drives many indicators

$$y_{i,t} = \lambda_i f_t + \varepsilon_{i,t}$$

each indicator loads on a common factor

$$f_t = A_1 f_{t-1} + \dots + \eta_t$$

the factor (state) evolves

The structure

- A few common factors capture the economy's state.
- Each indicator is that factor times a loading, plus noise.
- The factor evolves over time like a small VAR.

Why a factor

- Dozens of indicators share one underlying signal.
- The factor IS the hidden 'state of the economy'.
- Estimating it filters out each series' idiosyncratic noise.

Casting It in State Space

The factor is the state; the filter does the rest

ragged edge: if $y_{i,t}$ not yet released $\Rightarrow$ skip its update (treat as missing)

The mapping

- The common factor becomes the hidden state.
- Each indicator is an observation of it (with its loading).
- Mixed frequencies fit as observations at different steps.

Ragged edge, dissolved

- A not-yet-released value is simply a missing observation.
- The filter skips its update and moves on.
- No imputation, no truncation — it just works.

The Nowcast, Defined

The best estimate of the present

$$\text{nowcast} = \mathbb{E}[\text{GDP}_{\text{current}} \mid \text{all data released so far}]$$

What it is

- The expected current-quarter GDP given all data so far.
- It is a filtered estimate — causal, using only released data.
- It comes with an uncertainty band, like any filter output.

How it behaves

- Early in the quarter it is uncertain — little data yet.
- It sharpens as more indicators release.
- It converges toward the eventual official print.

The Nowcast Updates in Real Time

A living estimate, revised on every release

The nowcast is not a one-shot forecast — it is a running estimate that revises each time a new figure is published. As the quarter fills in, the point estimate settles and its uncertainty band narrows.

Through the quarter

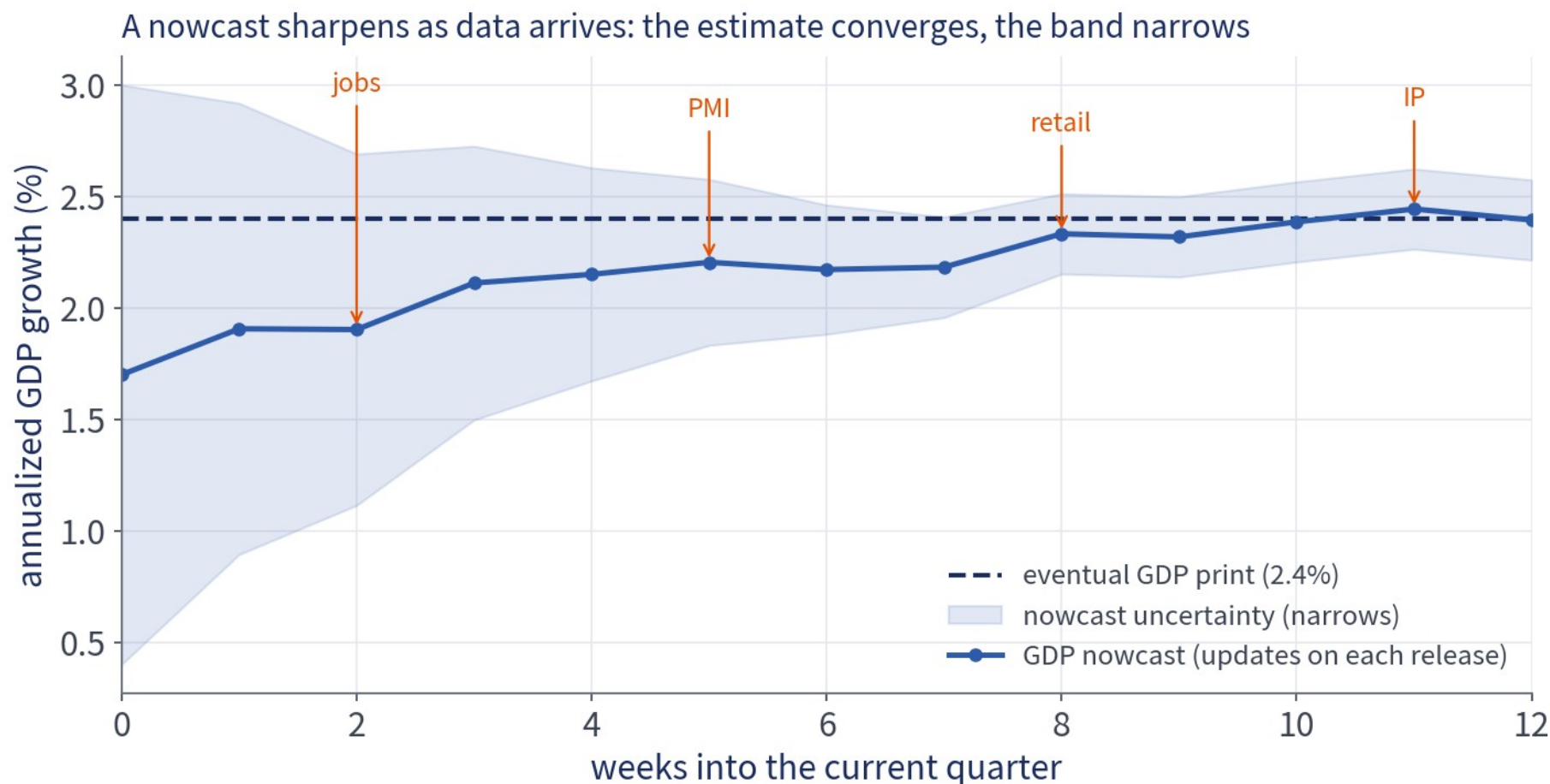
- Start from a prior with a wide uncertainty band.
- Each release nudges the estimate and tightens the band.
- By quarter's end it is close to the true value.

What this gives policymakers

- A continuously current read on growth.
- An early warning when momentum shifts.
- A principled way to weigh conflicting new data.

A Nowcast Sharpening Through the Quarter

Each data release (jobs, PMI, retail, industrial production) updates the nowcast. The point estimate converges toward the eventual GDP print and the uncertainty band steadily narrows.



'The News': Each Release Is a Surprise

The nowcast revision is an innovation

$$\text{nowcast revision} = K_t \times \underbrace{(\text{release} - \text{its forecast})}_{\text{the 'news' = innovation } v_t}$$

The connection

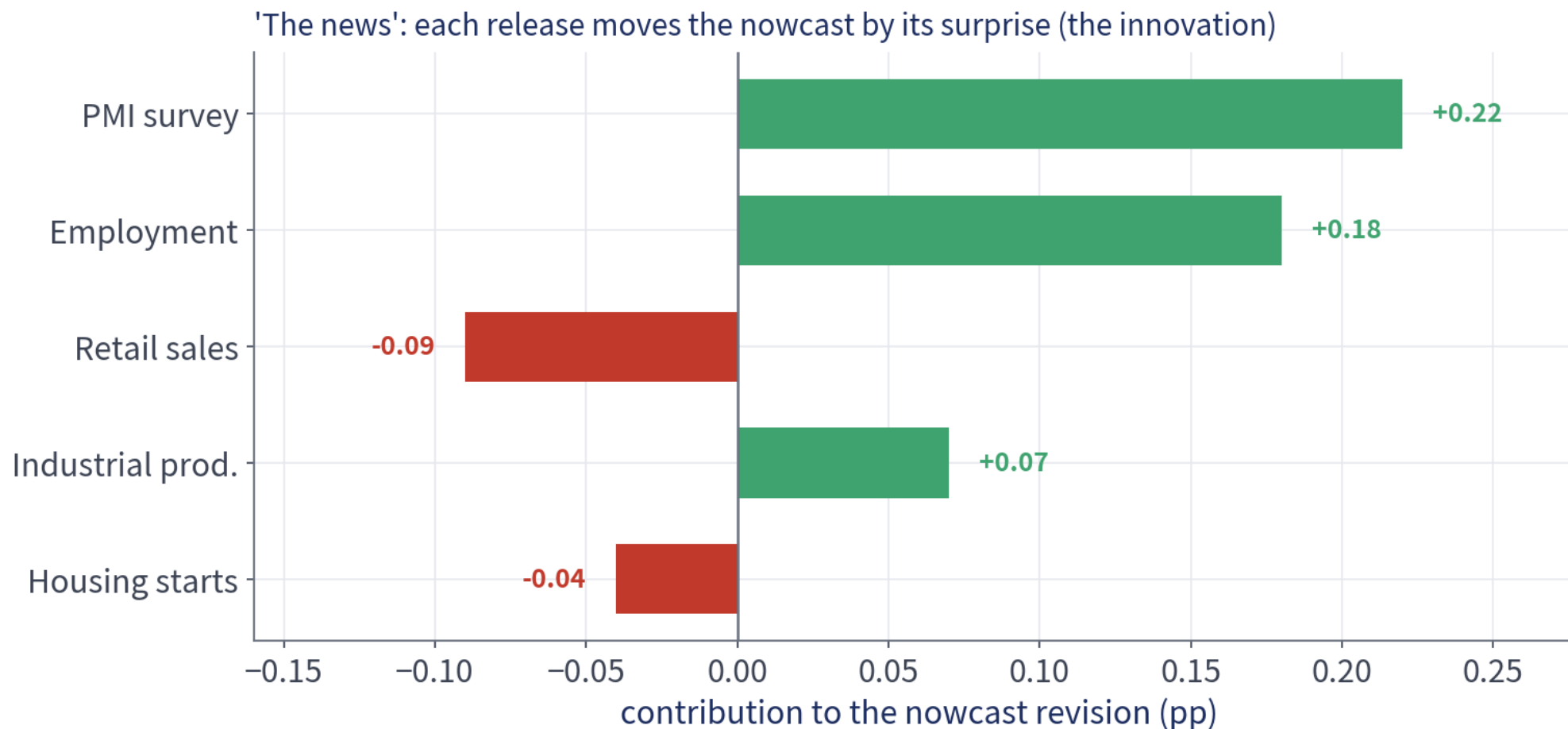
- A release matters only to the extent it SURPRISES.
- That surprise is exactly the Kalman innovation from Part 1.
- The revision is the gain times that surprise.

Why practitioners love it

- You can attribute each nowcast move to a specific release.
- An in-line number moves the nowcast little; a shock moves it a lot.
- This 'news' decomposition is a standard nowcasting output.

Decomposing the Nowcast Into News

Each recent release contributes to the nowcast revision by its own surprise — positive (green) or negative (red). The filter turns a stream of data into an interpretable set of contributions.



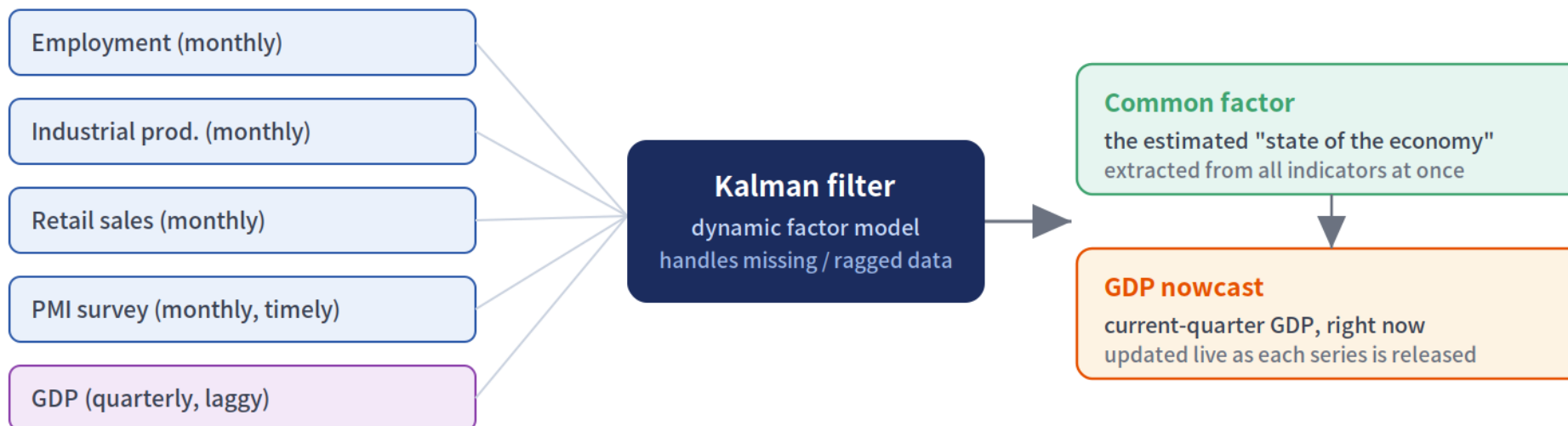
Nowcasting With the Kalman Filter

Many mixed-frequency, ragged-edge indicators feed one filter, which extracts the common factor — the state of the economy — and reads off a current-quarter GDP nowcast, updated on every release.

Nowcasting With the Kalman Filter

Many mixed-frequency, ragged-edge indicators feed one filter, which reads the state of the economy now.

Indicators (mixed frequency)



Each release is 'news'

Every new data point updates the nowcast by its surprise - the Kalman innovation. This is the same engine as Parts 1-2.

Nowcasting in the Real World

From a 2008 paper to central-bank dashboards

The literature

- Giannone, Reichlin and Small (2008) framed the method.
- Banbura, Giannone and Reichlin extended it to large panels.
- It rests on dynamic factor models estimated by the Kalman filter.

The practice

- Central banks publish live GDP nowcasts (e.g. Federal Reserve).
- The Atlanta Fed's GDPNow is a widely-watched example.
- Markets track these nowcasts around major data releases.

Pairs Trading vs Nowcasting

The same machinery, two problems

Aspect	Pairs trading (Ex. 1)	Nowcasting (Ex. 2)
Hidden state	a drifting hedge ratio	the state of the economy (a factor)
Observations	two asset prices	many mixed-frequency indicators
Key filter feature	tracking a drifting parameter	handling missing / ragged data
Output	a centered, tradable spread	a real-time GDP estimate

Different fields, one recursion: a hidden state, noisy observations, and predict-update.

A Word of Caution

The filter is only as good as its inputs

In both applications the filter is optimal ONLY if the model and noise assumptions hold. A wrong model or badly chosen noise levels will produce a confident but wrong answer — so always check the innovations and validate out of sample.

What to watch

- Innovations should look like white noise — no leftover structure.
- Choose the process and measurement noise with care.
- Validate on held-out data, not just in-sample fit.

When to reach elsewhere

- Strong nonlinearity → extended or unscented filters.
- Discrete, switching regimes → hidden Markov models (Week 10).
- Heavy-tailed shocks → robust or particle filters.

Build the Stage III Pairs Trade

A self-adjusting hedge ratio, end to end

GOAL Estimate a time-varying hedge ratio with a Kalman filter and back-test the adaptive pairs trade.

[INPUT] Prompt · Claude Code

Write and run `pairs_stage3.py` that:

1. models the hedge ratio β_t as a Kalman state (dynamic regression of Y on X);
2. forms the ADAPTIVE spread $Y - \beta_t X$ and compares it to the STATIC-beta spread;
3. z-scores the adaptive spread and back-tests: enter at $|z| \geq 2$, exit near 0, with a stop;
4. plots β_t , both spreads, and cumulative P&L for static vs dynamic.

Report which hedge keeps the spread centered.

How to work with it

- β_t should drift smoothly, not jitter.
- The adaptive spread should stay centered.
- Check innovations are white noise.
- This completes the pairs-trading thread.

Deliverable: `pairs_stage3.py` + a dynamic beta, both spreads, and a static-vs-dynamic back-test.

Build a Simple GDP Nowcaster

A dynamic factor model with ragged data

GOAL Fit a small dynamic factor model by Kalman filter and produce a nowcast that updates as data arrives.

[INPUT] Prompt · Claude Code

Write and run `nowcast_demo.py` that:

1. builds a few monthly indicators driven by one common factor, plus a quarterly GDP target;
2. fits a dynamic factor model in state-space form (`statsmodels DynamicFactor` or a hand-coded Kalman filter) allowing MISSING values;
3. introduces a ragged edge (drop the most recent values of the laggier series);
4. produces the nowcast and its band, and shows it update as each 'release' is added back.

Report how the nowcast and band change per release.

How to work with it

- Missing values must be allowed in the filter.
- The band should narrow as data fills in.
- Each release should move the nowcast by its surprise.
- This is nowcasting in miniature.

Deliverable: `nowcast_demo.py` + a nowcast that updates and tightens as releases arrive.

Three Questions Before You Finish

If you can answer these, Week 9 has landed

Q1	In Stage III, what is treated as the hidden state?	The drifting hedge ratio
Q2	How does the Kalman filter handle a not-yet-released indicator?	Skip its update (missing)
Q3	A nowcast jumps on a data release. What moved it?	The surprise (innovation)

All three come from the dynamic hedge, ragged-edge handling, and the news decomposition.

Week 9 in Three Ideas

Pulling the whole week together

1

Predict, update, and the gain

Parts 1-2: the two-step recursion, the innovation that drives it, and the Kalman gain that optimally weighs model against data — the whole filter in five equations.

2

One recursion, two flagship applications

Part 3: a drifting hedge ratio for pairs trading (Stage III), and a real-time GDP nowcast from ragged, mixed-frequency data. Both treat the target as a hidden state.

3

The same ideas, everywhere

Missing data is just a skipped update; a data 'surprise' is an innovation; a drifting parameter is a state. Master the recursion and these applications follow.

Next — Week 10: Hidden Markov Models, for a hidden state that switches between discrete regimes.

★ WEEK 9 · PART 3 · PRACTICE

One Filter, Two Uses - Worked by Hand

An Adaptive Pairs Signal, and a Nowcast Through Missing Data

The Kalman filter, applied to trading and to the economy

Part 3's applications, with the arithmetic shown. We turn a drifting hedge ratio into a live trading signal, then use a dynamic factor model to nowcast a hidden state of the economy - even when an indicator is missing.

3P

PART 3 · APPLICATIONS

Two Uses, Computed by Hand

Case A: an adaptive spread from a drifting hedge ratio, turned into a z-score and a signal - and why a fixed hedge fails. Case B: a nowcast from a dynamic factor model, filling the ragged edge of missing data. Then a Claude Code lab.

What Part 3 Told Us

Now we apply the filter - with numbers

1

The shared capability is denoising

Both uses pull a clean hidden state from noisy data - a fair hedge ratio, or the true state of the economy.

2

Pairs trading: an adaptive hedge

Let the hedge ratio drift as a state, and the spread stays mean-reverting. We turn it into a live signal by hand.

3

Nowcasting: a hidden factor

One factor drives many indicators arriving at different times. The filter reads the present, even with data missing.

Everything here is a calculator exercise: one spread and z-score, one factor from two indicators.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from the filtered hidden state.
- Compute the signal or the nowcast by hand.
- Then read the decision it supports.

What you should watch

- How an adaptive spread beats a fixed one.
- How a z-score becomes a trade.
- How the filter handles a missing indicator.

An Adaptive Pairs Signal

A drifting hedge, a live trade

In Week 8 we filtered a hedge ratio; now we trade on it. As the hedge ratio drifts, the filter keeps it current, so the spread stays mean-reverting. We compute today's adaptive spread, turn it into a z-score and a signal, and show that a fixed hedge ratio would have drifted off completely

The inputs

- Today's filtered hedge ratio beta = 0.90.
- Prices $A = 52.0$, $B = 57.0$.
- Recent spread mean 0.20, sd 0.25.

What we'll do by hand

- Form the adaptive spread and its z-score.
- Read off the trading signal.
- Compare against a stale fixed hedge.

The Adaptive Spread and Signal

A drifting hedge keeps the spread honest

$$\text{adaptive spread} = A - \beta_t B, \quad z = \frac{\text{spread} - \mu}{\sigma} \quad (\beta_t \text{ drifts, kept live by the filter})$$

The spread

- A minus the live hedge ratio times B.
- Because beta is current, the spread reverts.
- A fixed beta would let it wander.

The signal

- Standardize the spread into a z-score.
- Beyond +/- 2, the spread is stretched.
- Trade against the stretch, expect reversion.

Today's Spread and Signal

From a live hedge ratio to a trade

$$\text{spread} = 52.0 - 0.90(57.0) = 0.70$$

$$z = \frac{0.70 - 0.20}{0.25} = \mathbf{2.0} \quad \Rightarrow \quad |z| \geq 2 : \text{SHORT the spread}$$

The adaptive spread is 0.70, a z-score of exactly 2.0 - stretched two standard deviations rich, so the signal is to short the spread and wait for reversion.

Why a Fixed Hedge Fails

The same prices, a stale ratio

$$\text{fixed } \beta = 0.80 : \text{spread} = 52.0 - 0.80(57.0) = 6.40 \Rightarrow \text{miscalibrated, not tradable}$$

The fixed hedge

- An old beta of 0.80 gives a spread of 6.40.
- That's nowhere near the mean of 0.20.
- The z-score would be enormous - and useless.

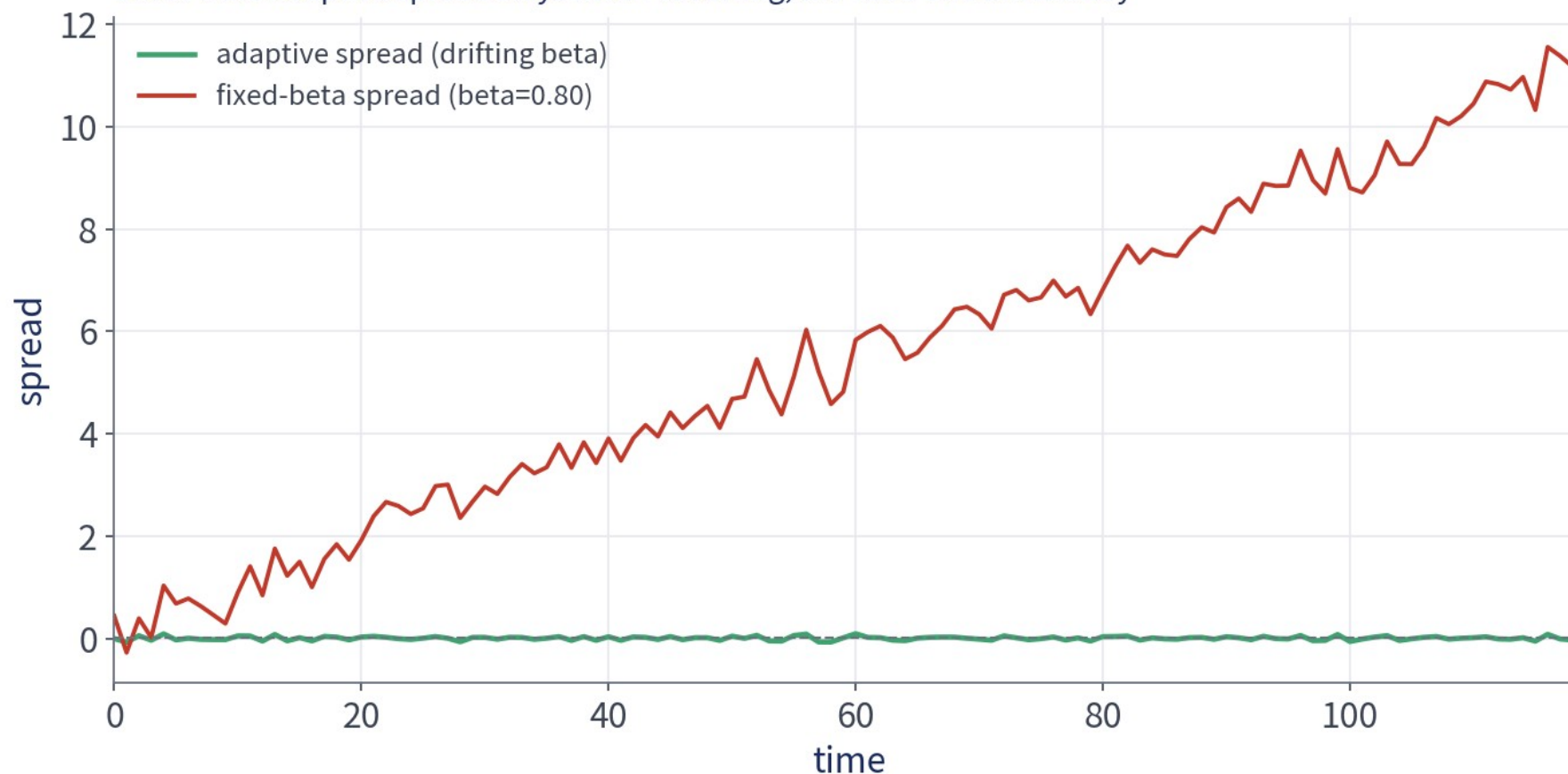
The lesson

- The relationship drifted; the fixed hedge didn't.
- Its spread is miscalibrated, not tradable.
- The live, filtered hedge keeps the trade honest.

Adaptive Stays Honest; Fixed Drifts

The adaptive spread (green) hugs its mean and oscillates - exactly what a pairs trade needs. The fixed-beta spread (red) drifts away as the hedge ratio changes, so its z-scores become meaningless. A live hedge keeps the signal tradable

Case A: the adaptive spread stays mean-reverting; the fixed one drifts away



The Filter Keeps the Trade Alive

Denoising, applied to a spread

Without the filter

- A one-time regression sets the hedge.
- It goes stale as the market shifts.
- The spread drifts, the signal breaks.

With the filter

- The hedge ratio updates every day.
- The spread stays centered and mean-reverting.
- The z-score remains a valid signal.

Nowcasting a Hidden Factor

Reading the present through missing data

The same filter reads the economy. A single hidden factor - the state of activity - drives many indicators that arrive at different times, so today some are missing (the ragged edge). The filter combines a prior with whatever indicators are available, dropping the missing one to produce a nowcast. We compute it by hand

The setup

- A hidden factor with persistence $\phi = 0.8$.
- Three indicators load on it (1.0, 1.2, 0.8).
- This period, indicator 3 is missing.

What we'll do by hand

- Predict the factor forward.
- Turn each available indicator into a read.
- Combine them into one nowcast.

The Dynamic Factor Model

One factor, many indicators

$$\begin{array}{ll} \text{state: } f_t = \phi f_{t-1} + \text{noise} & \text{(hidden factor)} \\ \text{observation: } y_{i,t} = \lambda_i f_t + \varepsilon_{i,t} & \text{(each indicator loads on } f) \end{array}$$

The state

- A hidden factor that persists over time.
- It's the common driver behind the data.
- The thing we want to nowcast.

The observations

- Each indicator is the factor times a loading.
- Plus its own measurement noise.
- Some arrive late or not at all.

Each Indicator Implies a Factor

Predict, then read the available data

$$\begin{aligned} \text{predict: } f^- &= \phi f_{t-1} = 0.8(0.30) = 0.24 \\ \text{indicator 1: } \hat{f} &= \frac{0.5}{1.0} = 0.50, \quad \text{indicator 2: } \hat{f} = \frac{0.7}{1.2} = 0.58, \quad \text{indicator 3: missing} \end{aligned}$$

Predict the factor forward to 0.24. Indicators 1 and 2 imply factor values of 0.50 and 0.58; indicator 3 is missing, so it simply drops out of this update.

Combine Into a Nowcast

Precision-weighted, missing dropped

$$\text{nowcast } f = \frac{\text{prior} \cdot p_0 + \hat{f}_1 p_1 + \hat{f}_2 p_2}{p_0 + p_1 + p_2} = \mathbf{0.47} \quad (\text{missing indicator simply drops out})$$

The combination

- Weight the prior and each read by precision.
- More precise reads count for more.
- The nowcast lands at 0.47.

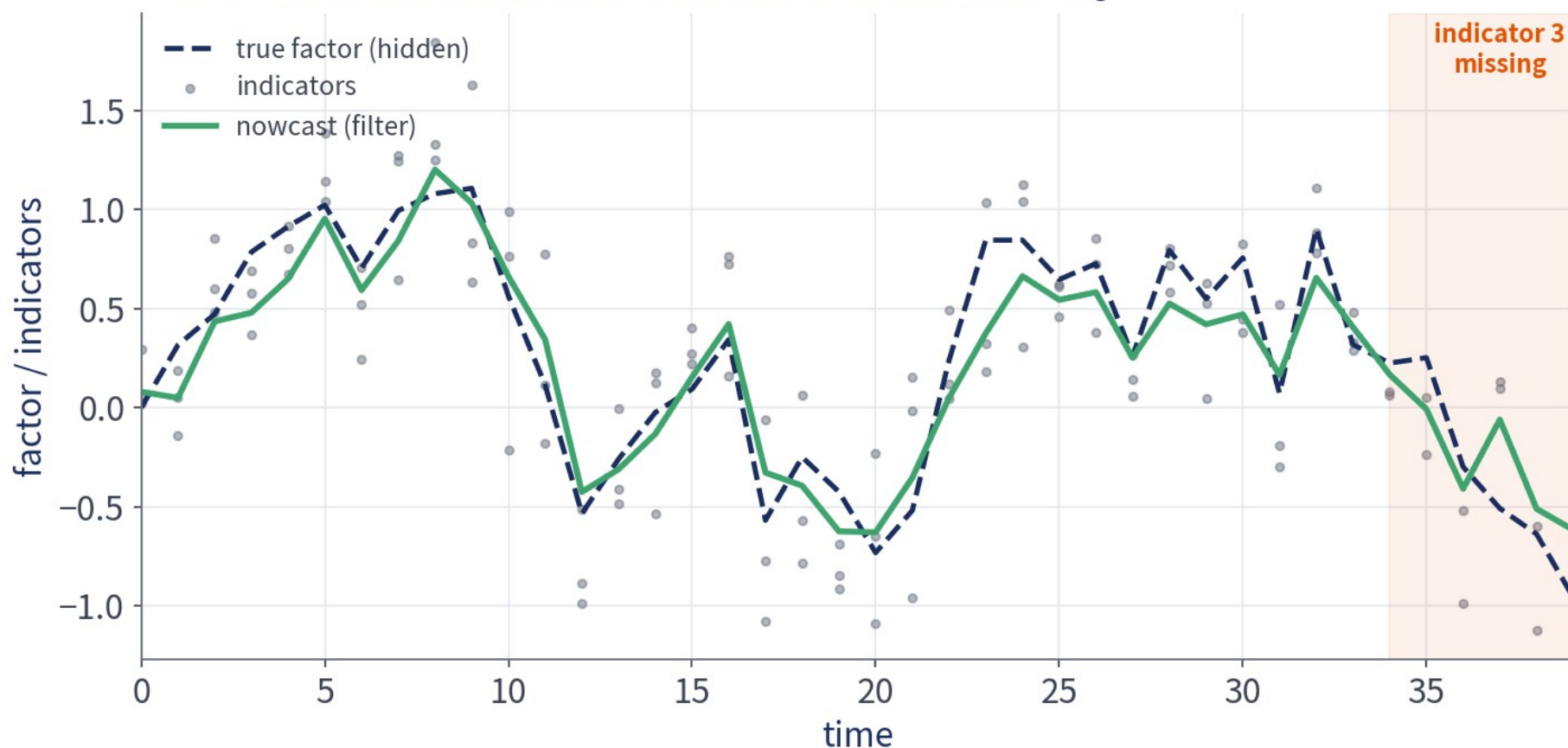
The ragged edge

- Indicator 3 contributes nothing this period.
- The filter uses only what's available.
- No imputation, no dropped update.

The Nowcast Tracks Through the Gap

The nowcast (green) follows the hidden factor (dashed) by pooling the noisy indicators. In the shaded stretch, indicator 3 is missing - yet the nowcast keeps tracking, because the filter simply weights whatever data has arrived. That is the ragged edge handled.

Case B: the nowcast tracks the factor - even where an indicator is missing



Why the Filter Fits Nowcasting

Missing data is not a problem

The hard part of nowcasting

- Indicators arrive weekly, monthly, quarterly.
- At any moment, some are not yet in.
- You still need a read on the present.

The filter's answer

- Update from whatever has arrived.
- Missing rows just drop out of the step.
- The nowcast is always current.

One Filter, Two Uses

The same engine, two problems

A pairs trade and a nowcast look nothing alike, yet both are the Kalman filter denoising a hidden state. In one, the state is a hedge ratio and the output is a trade; in the other, the state is the economy and the output is a real-time read. Same predict-update engine, different state

Pairs trading

- State: the drifting hedge ratio.
- Output: an adaptive spread and z-score.
- Value: a signal that stays valid.

Nowcasting

- State: the hidden factor of activity.
- Output: a real-time estimate of the present.
- Value: robustness to missing data.

Reproduce Both Applications

Direct Claude Code to check your arithmetic

GOAL Verify the adaptive signal and the nowcast, then run both on real data.

[INPUT] Prompt · Claude Code

Write and run `part3_practice.py` that:

CASE A - adaptive signal:

- * from $\beta=0.90$, $A=52.0$, $B=57.0$, print the spread, the z-score (mean 0.20, sd 0.25), and the signal;
- * compare to a fixed $\beta=0.80$ spread.

CASE B - nowcast:

- * predict the factor ($\phi=0.8$, prior 0.30);
- * from indicators 0.5, 0.7 (loadings 1.0, 1.2) with indicator 3 missing, print the nowcast.

Then run both on real series and plot the results.

Match every printed number to the slides.

Check against the slides

- adaptive spread 0.70, $z = 2.0$.
- fixed-beta spread 6.40.
- factor predict 0.24.
- nowcast 0.47.

Deliverable: `part3_practice.py` whose printed numbers match the signal and the nowcast.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 Why does an adaptive spread beat a fixed one?

The hedge drifts; the filter keeps it current

Q2 How does the filter handle a missing indicator?

It drops out; the update uses what's available

Q3 What do both applications have in common?

Denoising a hidden state, same filter

All three come from the adaptive-spread signal and the ragged-edge nowcast.

Three Pitfalls to Avoid

Where the applications slip

Common mistakes

- "A fixed hedge ratio is good enough."
- "Impute or drop the whole period if data is missing."
- "The nowcast needs every indicator."

The reality

- A drifting relationship needs a live hedge.
- The filter uses whatever has arrived - no imputation.
- A partial read still gives a valid nowcast.

Two Applications to Remember

Part 3 applications in one screen

1

An adaptive spread makes a live signal

With a filtered hedge ratio of 0.90, the spread was 0.70 and the z-score exactly 2.0 - a short signal. A stale fixed beta of 0.80 gave a spread of 6.40, miscalibrated and untradable. The filter keeps the trade honest.

2

A nowcast reads the present through gaps

A dynamic factor model predicted the factor to 0.24, then combined two available indicators (0.50 and 0.58) by precision into a nowcast of 0.47. The missing indicator simply dropped out - the ragged edge, handled.

3

One filter, two uses

Trading and nowcasting are the same predict-update engine denoising a hidden state - a hedge ratio, or the economy. Master the filter, and both applications follow.

Next — Week 10 and beyond: regimes and the hidden Markov model, where the state jumps rather than drifts.